



UNIVERSITÀ DI PISA

MASTER OF SCIENCE IN COMPUTER ENGINEERING

Large Scale and multi structured database project



Students:

Megan MAREMMANI

Eleonora SGORBINI

Chiara MASIERO

ACADEMIC YEAR 2025-2026

Abstract

Let yourself be inspired by the countless culinary ideas in our application. You can easily find the perfect dish by exploring thousands of recipes, filtering them by category, or browsing our 'Most Popular recipe per category' and 'Recently Added' sections. Looking for something specific? You can discover dishes written by your favorite chef or search directly by title and keywords.

At this point, you might think this sounds just like many other recipe platforms out there (though ours is undeniably better looking!). But picture this scenario: you are staring at your pantry, looking at dozens of seemingly unrelated ingredients, and you have absolutely no idea what to cook. All you need to do is enter them into your *Smart Fridge*. The app will instantly suggest sensational recipe ideas, helping you transform what you already have into delicious new meals.

If, on the other hand, you need to go grocery shopping but constantly lose your paper notes or end up making a hundred different lists, we have the perfect solution. Meet your new *Shopping List*: a digital companion you can always carry with you to write down everything you need to buy, ensuring you never forget an item at the supermarket again.

Finally, by accessing your personal profile, you can easily manage your favorite recipes. View them sorted by the date they were saved (from newest to oldest), and filter them by category and difficulty level.

In short, cooking has never been this fun and inspiring. What are you waiting for?

The code for the project are available on GitHub at the following link: <https://github.com/ItsMegan605/MySmartRecipeBook>

Contents

1	Introduction	2
1.1	Key Features	2
2	Application Design	3
2.1	Actors	3
2.2	Requirements	4
2.2.1	Functional Requirements	4
2.2.2	Non-functional Requirements	7
3	User Manual and Mockups	8
3.1	Home page	8
3.1.1	Search by Title	9
3.2	Recipe by category	10
3.3	Chefs search page	11
3.4	Chefs recipes page	11
3.5	Smart Shopping List	12
3.6	Smart Fridge	13
3.7	Login/Register	14
3.8	Foodie's Personal Area	15
3.9	Chef's Personal Area	16
3.9.1	Create a New Recipe	18
3.10	Admin's Personal Area	19
3.11	Recipe details view	20
4	UML class diagram	22
5	Dataset and Data Retrieval	23
5.1	Recipe Data Extraction	23
5.1.1	Scraping Process	23
5.1.2	Data Preprocessing	24
5.2	Foodies and Chefs Generation	25
5.2.1	User Profile Generation	25
5.2.2	Chef Profile Generation	26
5.3	Dataset Statistics	26
6	DB Design	27
6.1	Volume consideration	27
6.2	Document DB - MongoDB	30
6.2.1	Collections	30
6.3	Key-Value DB - Redis	37
6.3.1	Structure	37
6.3.2	Keys TTL	40

6.4	Graph DB - Neo4j	40
6.4.1	Graph Nodes	41
6.4.2	Graph Relationships	41
7	Implementation	44
7.1	Spring	44
7.2	Security	44
7.3	Config	46
7.4	Event	47
7.5	Model	47
7.5.1	MongoDB Models	47
7.5.2	Neo4j Models	54
7.6	Repository	56
7.7	Service	56
7.8	Controller	60
7.9	Endpoints	62
7.10	Utils	65
7.10.1	conversionFunctions	65
7.10.2	Parameters	65
7.10.3	Database Populators	65
8	Relevant queries	67
8.1	MongoDB Queries	67
8.1.1	Chef Ranking using Bayesian Score	67
8.1.2	Category Trend Analysis	69
8.1.3	Monthly Foodie Registration Statistics	71
8.1.4	Most Saved Recipe per Category	72
8.2	Neo4j Queries	73
8.2.1	Smart Fridge Recipe Suggestion	73
8.2.2	Similar Chefs by Shared Ingredients	75
8.2.3	Similar Recipes	76
8.2.4	Suggested Complementary Ingredients	77
8.3	Redis Queries and Caching Strategy	79
8.3.1	Key Patterns	79
8.3.2	Ingredients Management	79
8.3.3	Shopping List Management	80
8.3.4	Smart Fridge Management	82
8.3.5	Recipe Suggestions Caching	83
9	Design choices	86
9.1	Indexes	86
9.1.1	MongoDB Indexes	86
9.1.2	Recipes Collection	86
9.1.3	Chef Collection	89
9.1.4	Foodie Collection	91
9.2	Neo4j Graph Database Indexes	91
9.2.1	ingredient_name_idx	91

9.2.2	recipe_mongo_id_idx	93
9.2.3	chef_mongo_id_idx	94
9.3	Sharding	96
9.3.1	MongoDB sharding	96
9.3.2	Redis sharding	97
9.4	Virtual Machine Organization	97
9.4.1	Infrastructure Structure	97
9.4.2	Database VM Configurations	97
9.5	CAP Theorem Considerations	99
9.6	Handling Inter-DB Consistency	100
10	Use of Artificial Intelligence in Development	103
10.1	Overview	103
10.1.1	Web Scraping	103
10.1.2	Support in Database Population	103
10.1.3	Support in Mockups Creation	103
10.1.4	Java	104
10.1.5	Support in Translation	104
10.1.6	Redis	104
10.1.7	MongoDB and Neo4j	105

Chapter 1

Introduction

MySmartRecipeBook is a modern application designed for food enthusiasts and everyday home cooks eager to explore dishes created by professionals. The primary goal of the platform is to simplify, accelerate, and enhance the cooking process by providing an extensive, high-quality recipe catalog.

The platform is built around three main types of users: *Foodies*, who seek to discover new meals and optimize their daily grocery management; *Chefs*, who utilize the platform to unleash their culinary creativity and publish original recipes; and the *Admin*, who oversees the ecosystem to ensure high-quality standards and a secure, certified environment.

Ultimately, the system is designed to streamline the entire meal preparation experience, making cooking more engaging, fun, and accessible for all users.

1.1 Key Features

The main key features of the application are:

- **Advanced Search:** Foodies can browse thousands of recipes, utilizing filters such as category, chef's name, or specific keywords in the title, and seamlessly save their favorite dishes for future reference.
- **Smart Fridge:** This feature allows users to input available ingredients from their pantry to instantly receive tailored recipe recommendations, promoting culinary creativity and reducing food waste.
- **Smart Shopping List:** The application includes a portable digital shopping list that allows users to keep track of necessary ingredients while at the supermarket, ensuring they have everything needed to prepare their recipes at home.
- **Application Analytics:** The platform provides insights into culinary trends, enabling the Admin to track monthly registrations, monitor the most popular chefs, and identify trending recipe categories.
- **Certified Content:** A dedicated administrative layer ensures a safe environment by verifying chef certifications and reviewing all submitted recipes before publication to maintain high-quality standards.

Chapter 2

Application Design

2.1 Actors

The main actors of *MySmartRecipeBook* are:

- **Unregistered User:** A visitor who has not yet joined the platform. They can explore public content by searching recipes by title or category, browsing the most recently added ones, view the most popular recipe for each category, looking up chefs by surname to view their published recipes. To unlock further features, they can register through the login page by providing their personal details.
- **Foodie:** A registered user with full access to the platform. Beyond the capabilities of an unregistered user, a Foodie manages a personal area to update their profile or delete their account, saves favorite recipes (sortable by date and filterable by category and difficulty), and relies on two dedicated tools: the Smart Fridge, which suggests recipes matching at least three of their available ingredients, and the Smart Shopping List, to organize grocery shopping.
- **Chef:** A registered user certified by the Admin to ensure professional standards. A Chef manages their account details and authors recipes, which are not published immediately but enter a pending state awaiting Admin approval to guarantee content quality. Chefs can view their recipes sorted by publication date, identify their most popular ones (those exceeding a threshold of 40 saves), and delete recipes whether pending or already published.
- **Admin:** The system administrator who safeguards the platform's quality. The Admin approves or rejects pending recipe submissions, reviews and decides on Chef registration requests, and monitors platform analytics such as the monthly Foodie registration rate, chef rankings, and trending recipe categories.

2.2 Requirements

In the following section are listed the requirements of the application.

2.2.1 Functional Requirements

Unregistered User

- The system must allow an Unregistered User to search for recipes by title (including partial string matches) and browse the resulting matches through a paginated list.
- The system must allow an Unregistered User to browse recipes filtered by a specific category.
- The system must allow an Unregistered User to retrieve the complete list of recipes authored by a specific chef, sorted in descending chronological order.
- The system must allow an Unregistered User to retrieve the best recipe for each single category within the application.
- The system must display a preview card for each recipe in the browsing lists, explicitly containing the title, image, chef, and the total number of saves.
- The system must allow an Unregistered User to view the selected recipe's details, such as a brief presentation, difficulty, estimated preparation time, category, ingredients, step-by-step preparation, etc.
- The system must allow an Unregistered User to create a Foodie account by supplying a unique username, first name, last name, email, password, and date of birth, ensuring that the user is at least 15 years old.
- The system must allow an Unregistered User to create a Chef account by supplying a unique username, first name, last name, email, password, and date of birth, ensuring that the user is at least 15 years old.

Foodie

The Foodie role implicitly inherits all the functional requirements defined for the Unregistered User. In addition to those, the specific Foodie requirements are:

- The system must allow a user to login as a Foodie by providing a valid username and password.
- The system must allow a Foodie to access and view their personal profile page.
- The system must allow a Foodie to update their account information, specifically first name, last name, email, date of birth, and password.

- The system must allow a Foodie to delete their account.
- The system must allow a Foodie to search for chefs by specifying their surname, returning a list of matching profiles.
- The system must allow a Foodie to discover chefs with a similar culinary style to a specific chef, based on the ingredients they commonly use.
- The system must allow a Foodie to retrieve a list of three recipes similar to a selected one, based on shared ingredients.
- The system must allow a Foodie to save a recipe.
- The system must allow a Foodie to view their saved recipes, sorted by default in descending chronological order.
- The system must allow a Foodie to filter their saved recipes by a specific category or difficulty level.
- The system must allow a Foodie to add ingredients to their own smart fridge.
- The system must allow a Foodie to remove ingredients from their own smart fridge.
- The system must allow a Foodie to view the contents of their smart fridge alongside the dynamically suggested recipes.
- The system must allow a Foodie to add required ingredients to their shopping list.
- The system must allow a Foodie to remove ingredients from their shopping list.
- The system must allow a Foodie to view their shopping list.
- The system must allow a Foodie to receive suggestions for complementary ingredients based on the current items in their shopping list.

Chef

The Chef role implicitly inherits all the functional requirements defined for the Unregistered User. In addition to those, the specific Chef requirements are:

- The system must allow a user to login as a Chef by providing a valid username and password.
- The system must allow a Chef to access and view their personal profile page.
- The system must allow a Chef to update their account information, specifically email, date of birth, and password.
- The system must allow a Chef to delete their account.

- The system must allow a Chef to create a new recipe by providing its title, photo, brief presentation, category, difficulty level, preparation time, list of ingredients with the corresponding quantities, and step-by-step instructions.
- The system must allow a Chef to view a paginated list of their own published recipes, sorted in descending chronological order based on the creation date.
- The system must allow a Chef to retrieve a paginated view of their popular recipes (those exceeding the predefined threshold of 40 saves), ordered by save count in descending order.
- The system must allow a Chef to delete their own recipes from the ones already published on the platform.
- The system must allow a Chef to view their queue of pending recipes, sorted in ascending chronological order based on the submission date.
- The system must allow a Chef to view the full details of any recipe in their pending queue.
- The system must allow a Chef to delete their own recipes that are still waiting for Admin approval.

Admin

The Admin role implicitly inherits all the functional requirements defined for the Unregistered User. In addition to those, the specific Admin requirements are:

- The system must allow an Admin to authenticate into the platform by providing valid administrative credentials (username and password).
- The system must allow the Admin to view a list of pending Chef registration requests, sorted in ascending chronological order, displaying only the Chef's full name and username on the preview card.
- The system must allow the Admin to view the complete registration details of a pending Chef (first name, last name, date of birth, email, username) by selecting their respective preview card.
- The system must allow the Admin to approve a pending Chef registration request, officially granting them access to the platform.
- The system must allow the Admin to reject a pending Chef registration request.
- The system must allow the Admin to view a list of pending recipe submissions awaiting approval, sorted in ascending chronological order.
- The system must allow the Admin to view the full details of a pending recipe (title, preparation time, category, etc.) by selecting its corresponding preview card.

- The system must allow the Admin to approve a pending recipe.
- The system must allow the Admin to reject a pending recipe submission.
- The system must allow the Admin to access a dedicated dashboard to view platform analytics, specifically:
 - Monthly trend of new foodie registrations sorted by year, with descending months based on the number of subscriptions.
 - Chef leader-board based on a Bayesian ranking algorithm.
 - Popularity trends for each recipes' category.

2.2.2 Non-functional Requirements

- The system must be developed using an Object-Oriented Programming (OOP) language (Java).
- The system must implement a multi-database architecture utilizing a DocumentDB, a GraphDB, and a Key-Value DB.
- The system must securely encrypt the users' passwords.
- The system must be provided as a responsive website application.
- The system must be capable of handling high traffic effectively.
- The system must expose all its functionalities by means of well-documented RESTful APIs, in accordance with the OpenAPI specification.

Chapter 3

User Manual and Mockups

Before detailing the individual interfaces, it is important to note that all views within the application share a common top navigation bar, designed to ensure seamless and intuitive exploration of the platform.

Moreover, all recipe-related sections are paginated. Each page can display a maximum of five recipe cards and users can navigate between pages using the dedicated *Next* and *Previous* buttons. If some mockups show fewer than five recipes, this is only for illustrative purposes.

In addition, each recipe preview card is designed as an entirely clickable element that redirects the user to the comprehensive recipe details view, as detailed in Section 3.11. Similarly, the Chef's name within the card functions as a direct link leading to the detailed Chef's recipes page, which is described thoroughly in Section 3.4.

3.1 Home page

The *Home page*, accessible to all users, both registered and unregistered, serves as the main entry point to the application. It features a search bar designed to intuitively find recipes by title; this search functionality is highly flexible, allowing users to input even partial substrings to successfully retrieve a list of matching results.

Additionally, the Home page features a paginated section displaying the newest recipes published on the platform, sorted in descending chronological order.

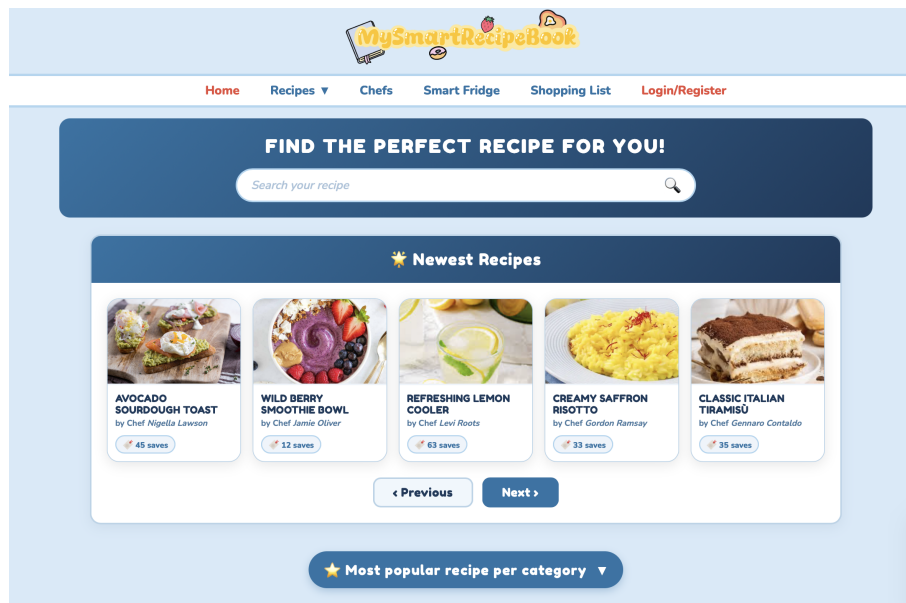


Figure 3.1: Home page of the application

Below this, an expandable section (hidden by default) can be toggled by the user to reveal a grid highlighting the single most popular recipe for each available category within the application. To ensure immediate clarity, each recipe card prominently displays a representative image of the dish, its title, the authoring Chef, and the total number of saves received from Foodies.



Figure 3.2: Home page with most popular recipe per category

3.1.1 Search by Title

When a user enters a keyword in the search bar, the system returns all recipes whose title contains the inserted text, without requiring an exact match. For example, by searching for the term *"pumpkin"*, the application retrieves all matching recipes containing that substring in their title. The resulting recipes are displayed as preview cards, maintaining the same structure used across the application: recipe image, title, chef's full name and number of saves.

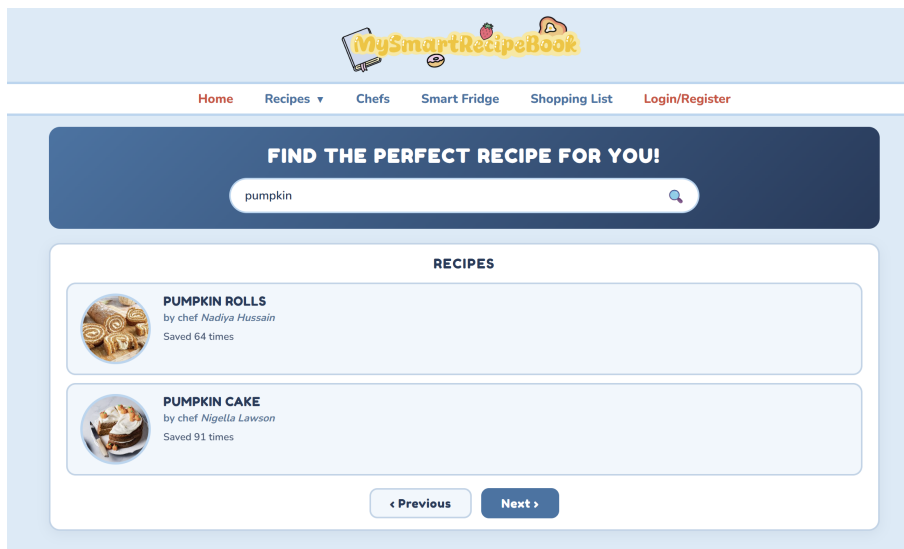


Figure 3.3: Recipe search results for "pumpkin"

3.2 Recipe by category

The *Recipes* item in the main navigation bar opens a drop-down menu of all available categories (e.g., Appetizers, Main Courses). Selecting a category dynamically filters the catalog, displaying the matching recipes in descending chronological order, from the newest to the oldest ones. Results are presented via preview cards as in the Home page.

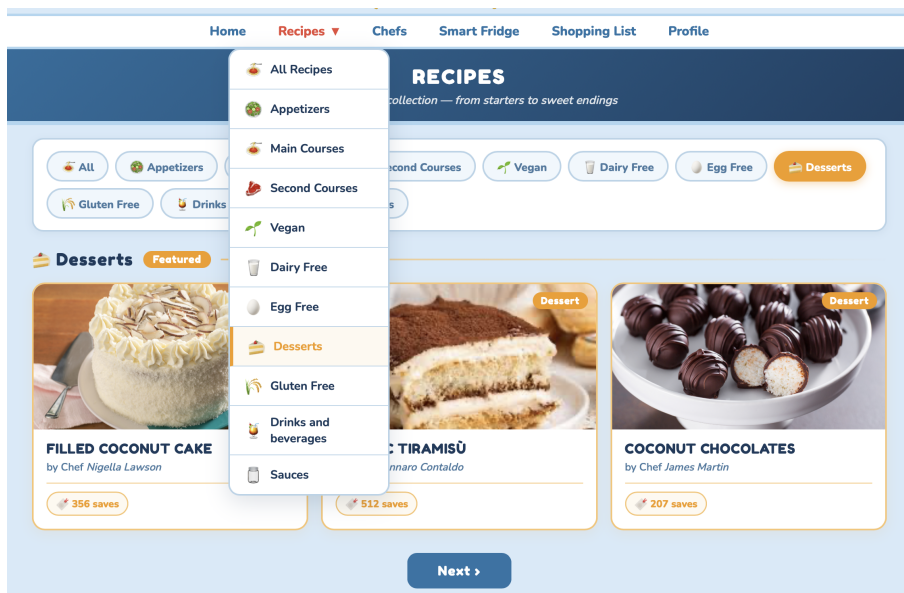


Figure 3.4: Recipes by category

3.3 Chefs search page

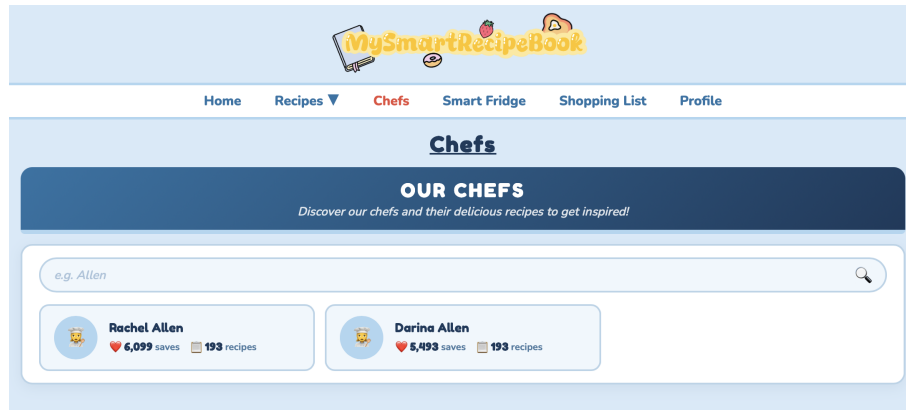


Figure 3.5: Chefs search page

The *Chefs search page* features a dedicated search bar allowing Foodies to look up specific authors by their last name. Upon entering the target string, the application processes the query and retrieves a grid of matching profiles. Each search result provides an overview of the chef, displaying their full name alongside the total number of published recipes and the cumulative amount of saves received from the Foodies. Within this results grid, the chef's name functions as an active link, directly redirecting the user to their dedicated recipes page, as described in the following Section 3.4.

3.4 Chefs recipes page

This dedicated view serves as the detailed portfolio for a specifically selected author (e.g., Nigella Lawson). The primary section of the interface showcases the comprehensive list of recipes published by the chef, automatically sorted in descending chronological order to highlight the latest additions.



Figure 3.6: Example of a Chef's profile and recipe list

Additionally, the profile page provides a *"See Similar"* functionality, exclusively available to registered users. To maintain a clean interface, this feature is hidden by default and can be expanded via a toggle button upon the user's request. Once activated, the system suggests a curated list of authors with a culinary profile comparable to the currently selected one, encouraging Foodies to explore new Chefs who match their specific interests.

This similarity score is algorithmically determined at a high level by analyzing the overlapping usage of ingredients across identical recipe categories. More details in section 8.2.2.

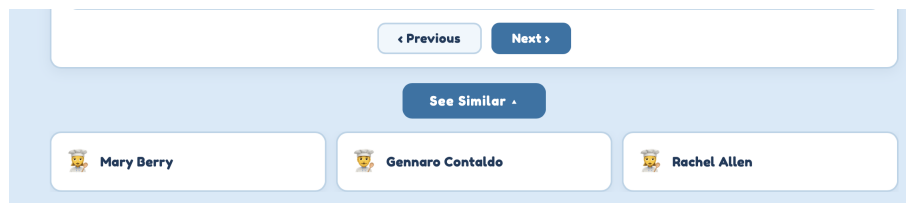


Figure 3.7: Example of the "See Similar" expandable section

3.5 Smart Shopping List

The *Smart Shopping List* is a practical tool designed for Foodies to manage their grocery needs efficiently. Within the list, ingredients can be removed one at a time by clicking the adjacent minus icon. Conversely, adding new items supports a seamless batch workflow: the front-end features an autocomplete system that guides the user's input, clearing the search bar and staging selected ingredients above it so

multiple items can be prepared sequentially. These staged ingredients are officially committed to the shopping list only after clicking the confirmation button.

In addition, the interface offers a personalized recommendation feature. Hidden by default to ensure a clear layout, the *"Suggested Ingredients"* panel expands only upon the Foodie's explicit request. Once opened, it displays complementary ingredients frequently paired with the items currently in the basket, grouping suggestions logically to help users complete their shopping list and discover optimal ingredient combinations. See in section 8.2.4.

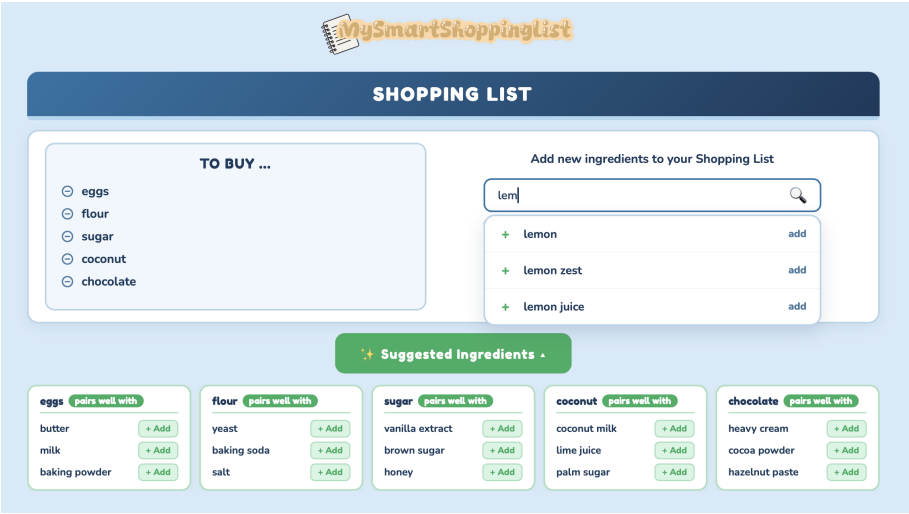


Figure 3.8: Smart Shopping List

3.6 Smart Fridge

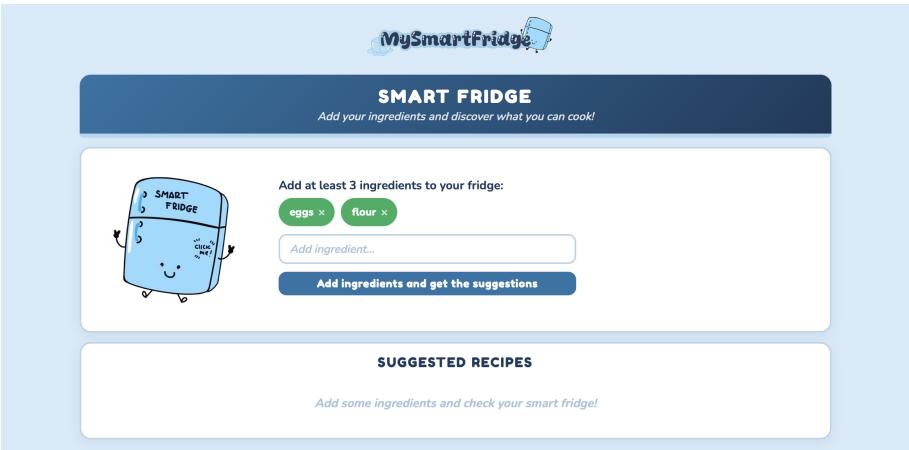


Figure 3.9: Smart Fridge before reaching the minimum number of ingredients

It is available exclusively to *Foodies*, the *Smart Fridge* combats food waste by offering recipe suggestions based on the user's virtual pantry. It shares the same

intuitive item management workflow as the Smart Shopping List.

In order to retrieve the recommendation, the system enforces a minimum threshold of 3 ingredients. Until this requirement is met, no suggestions are provided, and the lower section of the interface remains empty, as shown in Figure 3.9.

The screenshot displays the 'SMART FRIDGE' application interface. At the top, a dark blue header contains the title 'SMART FRIDGE' and the subtitle 'Add your ingredients and discover what you can cook!'. Below this, the interface is divided into three main sections. The first section on the left features a cartoon refrigerator character with the text 'SMART FRIDGE' on its door. To its right, under the heading 'Add ingredients to your fridge:', there are three green buttons labeled '+ lemon', '+ cream', and '+ milk'. Below these buttons is a text input field with the placeholder 'Add ingredient...' and a blue button labeled 'Add ingredients and get the suggestions'. To the right of this section is a box titled 'YOUR INGREDIENTS' containing a list of ingredients: eggs, flour, sugar, coconut, and chocolate, each preceded by a minus sign in a circle. The second section, titled 'SUGGESTED RECIPES', contains three recipe cards. Each card includes a representative image of the dish, the title, the chef's name, and a green box indicating the number of ingredients matched and a list of the matched ingredients. The first card is for 'FILLED COCONUT CAKE' by Chef Nigella Lawson, showing 4 ingredients matched (coconut, flour, eggs, sugar). The second card is for 'COCONUT PANCAKES' by Chef Monica Galletti, also showing 4 ingredients matched (coconut, flour, eggs, sugar). The third card is for 'COCONUT CHOCOLATES' by Chef James Martin, showing 3 ingredients matched (coconut, sugar, flour). At the bottom of the interface, there are two buttons: '< Previous' and 'Next >'. The interface is designed with a clean, modern aesthetic using a light blue and white color palette with green accents for buttons and highlights.

Figure 3.10: Smart Fridge

Once the threshold is reached, the system automatically retrieves compatible recipes. Results are ranked in descending order based on the number of matched ingredients, prioritizing dishes that make the most efficient use of the user's available pantry.

These results are displayed through dedicated recipe cards designed to provide immediate clarity. Each card includes the title, a representative image of the dish, the Chef's name, the total count and explicit list of the matched ingredients.

3.7 Login/Register

The Login/Register page allows users to authenticate or create a new account by selecting their specific role.

The registration form requires personal information (username, password, email, first name, last name, and birth date). The system enforces global username uniqueness across both *Foodies* and *Chefs*, displaying an error message if the chosen username is already taken and an age check as well.

Although the input fields are identical for both roles, the interface provides two distinct tabs due to different backend validation workflows. Once verified, Foodie

accounts are created and activated immediately. Conversely, chef registrations enter a pending state requiring explicit Admin approval, a necessary step to certify the chef's identity and maintain the platform's high quality standards.

Figure 3.11: Registration page

Figure 3.12: Login page

Regarding the login process, any user wishing to access the platform, whether a Foodie or a Chef, must provide their registered username and password. The system automatically performs all necessary authentication checks to verify the entered credentials, ensuring secure access before redirecting the user to their customized personal area.

3.8 Foodie's Personal Area

Upon successful login, the Foodie unlocks full access to the platform's exclusive services, including the Smart Fridge, the Shopping List and the possibility to save their favorite recipes. The Personal Area is organized into three distinct sections to ensure clear and intuitive profile management.

The top section displays a summary of the user's registration information (for security reasons, the password is intentionally obscured). Foodies can also find a *"Modify"* button on the front, so that they can easily modify their personal information. Additionally, on the right side of this panel, there are two quick-access buttons (*"View your ShoppingList"* and *"View your SmartFridge"*) acting as convenient shortcuts to the application's interactive tools.



Figure 3.13: Foodie personal area

The second section hosts all the recipes the Foodie has saved, displayed by default in descending chronological order (from the recently saved to the oldest). To easily navigate them, the user can apply filters via a dedicated dropdown menu, isolating recipes based on specific categories or difficulty levels. Each recipe card also includes a *"Remove"* button to enable Foodies to remove a recipe from his saved ones. Finally, the page terminates with an area dedicated to permanent profile deletion.

3.9 Chef's Personal Area

Upon successful authentication, the Chef accesses a dashboard logically divided into functional sections to manage both their profile and culinary portfolio.

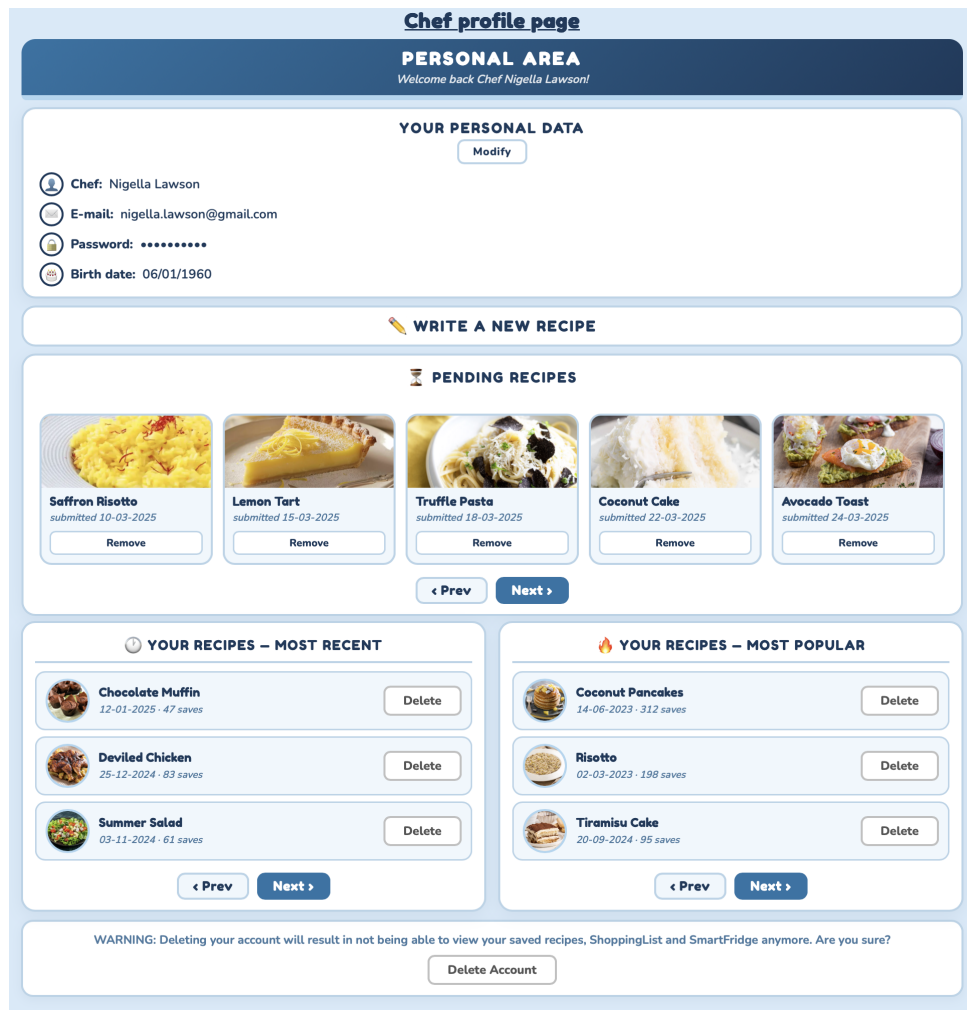


Figure 3.14: Chef's profile view

The top section summarizes account details (with an intentionally obscured password) and provides a *"Modify"* button for personal updates. Just below, the *"WRITE A NEW RECIPE"* button allows chefs to submit new content. To guarantee platform quality and safety standards, submitted recipes are not published immediately but are placed in a mandatory Admin review queue.

The *"PENDING RECIPES"* section displays these awaiting submissions as clickable, paginated cards. Chefs can review their contents or easily withdraw a submission using the designated *"Remove"* button.

The bottom area shows the Chef's published portfolio, divided in:

- *Your Recipes – Most Recent*: Published recipes sorted in strict descending chronological order.
- *Your Recipes – Most Popular*: Successful creations that have reached a minimum threshold of 40 user saves, sorted by total save count.

In both subsections, recipes are presented as clickable cards that redirect to the full recipe view. Each entry includes a "Delete" button, granting the Chef complete control to remove their published content from the platform at any time. Finally, the page terminates with an area dedicated to permanent profile deletion.

3.9.1 Create a New Recipe

When a Chef chooses to write a new recipe, they are directed to a dedicated creation form. To guarantee the high quality, completeness, and uniformity of the platform's content, all fields within this interface are strictly mandatory:

- *Recipe details*: The exact title, a short presentation, step-by-step preparation instructions, and a valid image upload to visually represent the dish.
- *Classification*: The recipe's category and difficulty level (both selectable via predefined dropdown menus), along with the estimated preparation time.
- *Ingredients*: A detailed list of required components. The Chef must select each ingredient from a predefined dropdown list (the same as the Shopping List and the Smart Fridge) and specify the corresponding quantity (e.g., "100g").

Figure 3.15: New Recipe interface

Once the form is fully populated, the Chef can submit the draft by clicking the "Save recipe" button. Upon submission, the platform executes a series of rigorous background validations: first of all, the backend strictly verifies that the selected category, difficulty, and ingredients belong to the platform's internally allowed and

predefined values.

If any of these validation checks fail, the submission process is immediately halted, and an explicit error message is displayed to guide the Chef in correcting the issues. Conversely, if all checks are passed, the interface provides immediate visual feedback via a confirmation popup, informing the Chef that their recipe has been correctly routed to the "Pending Recipes" section for the Admin's final review.

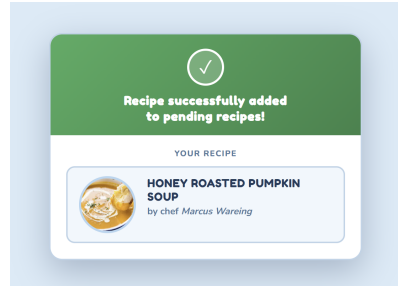


Figure 3.16: Confirmation popup after submitting a new recipe

3.10 Admin's Personal Area

To enable the Admin to effectively monitor and preserve platform quality, the dashboard is structured into two distinct parts: a moderation area at the top and an analytics area at the bottom.

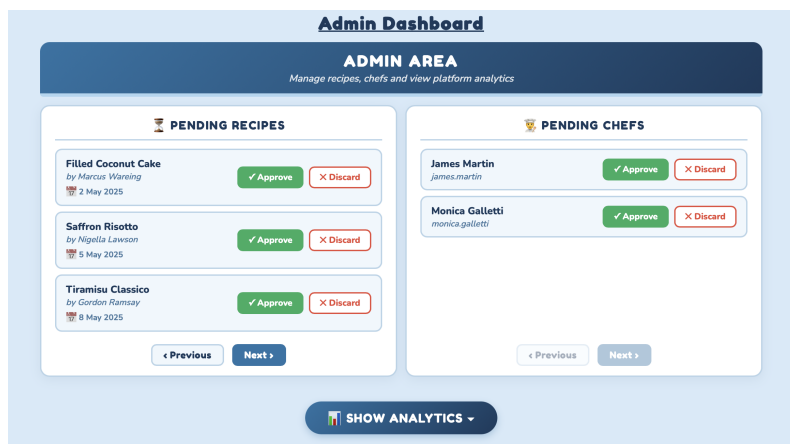


Figure 3.17: Admin profile page

The upper section focuses on content and user validation, featuring two operational panels:

- *Pending Recipes*: Lists recipes awaiting approval. Each card displays the title, the Chef's full name, the submission date and a representative image.
- *Pending Chefs*: Lists of chef registration requests, each containing the applicant's full name and username.

Both types of cards are fully clickable, allowing the Admin to inspect the comprehensive details of each pending request. Specifically, interacting with a card reveals the complete recipe description in the first case, and the full personal data of the applicant (such as email and date of birth) in the second.

The Admin can process these queues directly using the *"Approve"* button to publish recipes or validate Chef accounts, and the *"Discard"* button to reject them.



Figure 3.18: Admin Analytics view

To maintain a clear interface, the lower analytics section is hidden by default. It can be expanded upon the Admin's request by interacting with the *"Show Analytics"* toggle button. Once opened, it offers a data-driven overview of platform performance through three distinct panels:

- *Monthly Foodies*: A bar chart tracking yearly Foodie registration trends.
- *Chef Ranking*: A leaderboard of top Chefs, ordered by the Bayesian ranking algorithm.
- *Category Trends*: A dynamic overview showing percentage variations in user interest across recipe categories.

3.11 Recipe details view

As anticipated in the introduction, the *Recipe details* view serves as the primary destination when interacting with any recipe card, aggregating all the essential information required to prepare the dish. The interface prominently displays general useful information, including the author, category, difficulty level, and estimated preparation time. It also features a button to save the recipe to the user's favorites, a functionality exclusively reserved for Foodies. The layout is completed by a

brief presentation of the dish, followed by the detailed ingredients list with the corresponding quantities and step-by-step preparation instructions.

Recipe

FILLED COCONUT CAKE

by Chef Marcus Wareing



Filled Coconut Cake

by Chef Marcus Wareing · published 02/11/2019

Desserts

Average

60 min

Save recipe

You're craving a filled coconut cake with that Italian flair, huh? Well, this recipe's got you covered. It totally embraces the classic Italian vibe with its simple, moist goodness and a touch of class — perfect for those special moments. We're talking a cake with a yogurt-infused batter and shredded coconut that comes out so tender, giving you a base that's really flavorful. And the filling? A creamy mix of mascarpone and cream cheese. Seriously good. It adds this smooth, fresh burst that turns a simple cake into something truly special.

INGREDIENTS

Type 00 flour	2 ½ cups (300 g)
Sugar	1 ¼ cup (250 g)
Coconut yogurt	¾ cup (170 g)
Coconut flakes	1 ½ cup (135 g)
Sunflower seed oil	8.8 oz (250 g)
Milk	½ cup (125 g)
Eggs	1
Egg whites	1

PREPARATION

To prepare the filled coconut cake, first pour the coconut yogurt, milk, vegetable oil, and egg into a bowl. Add the egg white, lemon juice, and white wine vinegar. Mix until you obtain a homogeneous mixture. In another bowl, sift the flour and the baking soda, then add the sugar and shredded coconut. Mix well, then incorporate the dry ingredients into the liquid ones. Blend everything with a hand whisk until you get a smooth and homogeneous batter.

Butter two 8-inch springform pans, place a circle of parchment paper on the bottom, then flour the edges. Divide the batter equally between the pans. Place both pans on the lower rack of the oven and bake in a preheated static oven at 350°F for 35 minutes. During the last few minutes, check the cooking by doing the toothpick test. Once baked, remove from the oven and let cool completely.

Figure 3.19: Detailed recipe page

At the bottom of the page, Foodies can expand a dedicated *"See Similar"* section. This area suggests alternative dishes that share ingredients with the currently viewed recipe. Each recommendation is presented as a standard clickable card that additionally highlights the specific count and the explicit list of matching ingredients, helping Foodies seamlessly discover new culinary options related to their interests. See section 8.2.3 for details on this query.

Cream cheese1 ½ cup (375 g)

Icing sugar1.67 cups (200 g)

Coconut flakes (deco)0.67 cup (50 g)

Let the cake rest in the refrigerator for at least 2-3 hours before serving. Your filled coconut cake is ready!

See Similar



CLASSIC TIRAMISÙ

by Chef Gennaro Contaldo

3 ingredients matched ✓

eggs

sugar

cream



CHOCOLATE MUFFIN

by Chef Nigella Lawson

4 ingredients matched ✓

flour

eggs

sugar

milk



LEMON TART

by Chef Mary Berry

3 ingredients matched ✓

flour

sugar

cream

Figure 3.20: Suggested similar recipes

Chapter 4

UML class diagram

Figure 4.1 illustrates the UML class diagram for *MySmartRecipeBook*:

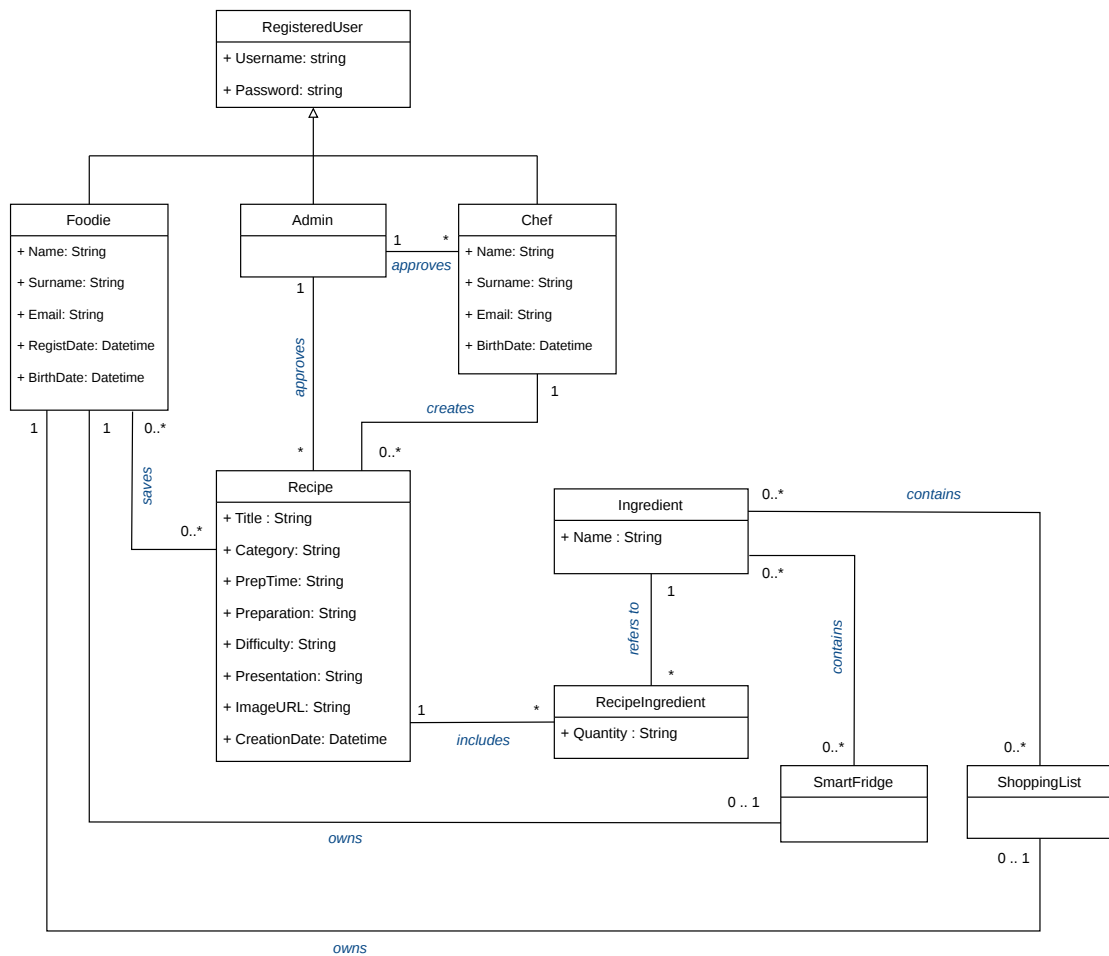


Figure 4.1: UML Class Diagram

Chapter 5

Dataset and Data Retrieval

The development of *MySmartRecipeBook* application required the acquisition, generation when needed and pre-processing of heterogeneous data related to users, recipes, and ingredients.

5.1 Recipe Data Extraction

As far as Recipes are concerned, publicly available datasets (from platforms like Kaggle) proved to be inadequate for our architectural needs. For instance, many existing datasets lacked essential metadata like culinary categories or failed to explicitly parse ingredient strings into separate "name" and "quantity" fields.

Consequently, to ensure the necessary data granularity and quality, we opted to construct a custom dataset by employing web scraping techniques, specifically targeting the extraction of recipe information. To acquire authentic, real-world data and ensure variety within our database, we chose to extract the required information from three distinct culinary websites:

- *BBC GoodFood*: A popular UK-based culinary website known for its professionally tested recipes. It provides high-quality data with precise measurements and a wide variety of international dishes.
- *AllRecipes*: One of the largest community-driven food platforms in the world. It offers a massive and diverse collection of everyday home-cooked recipes.
- *GialloZafferano*: The most famous Italian cooking portal. It is an excellent source for authentic Italian recipes that feature very detailed step-by-step preparation instructions.

5.1.1 Scraping Process

The extraction script was developed in Python, using standard libraries for web scraping and text processing. Because websites have different layouts, we created custom logic for each source to ensure accurate data retrieval. This allowed us to pull the relevant fields directly from the HTML, collecting the following details for every recipe:

- Title
- Image URL

- Brief introduction
- Preparation time
- Category
- Difficulty
- List of ingredients, separated by name and quantity
- Preparation steps

The data were then processed and saved in JSON format, making it easier to manipulate and combine the information collected from the different websites. It is also a format directly compatible with document-oriented databases such as MongoDB.

5.1.2 Data Preprocessing

After the retrieval phase, the collected data required several pre-processing activities in order to improve their quality and consistency, which includes the following steps:

- **Recipe deduplication**

Since we gathered data from the web, a deduplication step was necessary to clean the dataset. Duplicates mainly originated from two situations: recipes appearing on multiple pages of the same site with slightly different presentation texts or categories, and popular chefs publishing their dishes across different platforms, meaning the same recipe was collected from multiple sources.

- **Ingredient normalization**

Ingredient names varied significantly across the scraped websites. This was particularly evident on community-driven platforms that lacked strict input validation. As a result, the raw ingredient fields often contained embedded quantities, preparation instructions, typographical errors, and inconsistent singular/plural forms. A normalization procedure was applied to extract the core ingredient and standardize the dataset.

For example, embedded quantities and packaging details were removed:

`"1 (14- to 16-ounce package) frozen cauliflower tots" → "frozen cauliflower tots"`

Additionally, various preparation notes and states were stripped to group identical ingredients under a single standardized name. For instance, all the following raw inputs:

`"bell peppers (any color) stems and seeds removed, cut in half lengthwise"`

`"bell peppers - halved, seeded, and stems removed"`

`"bell peppers, any color"`

"bell peppers, chopped into 3/4 inch pieces"

"bell peppers, thinly sliced"

"bell peppers, tops cut off and seeded"

were all hand normalized to bell peppers.

- **Text cleaning**

Textual data extracted from HTML pages often contained formatting artifacts or unnecessary characters. For instance, strings of five or more consecutive newline characters ("`\n`"), likely used for HTML layout purposes, were identified and removed to ensure a clean and uniform text structure.

- **Missing data generation**

In cases where attributes such as difficulty, category, preparation time, or ingredient quantities were missing, the data was manually integrated. This refinement process involved re-consulting the original source to retrieve information that may have been misplaced (e.g., quantities incorrectly included in the ingredient name field). When necessary, missing values were deduced based on the recipe's context or by cross-referencing similar preparations on the web to ensure a complete and consistent dataset.

These pre-processing steps significantly improved the usability of the dataset and made it suitable for storage and analysis within the different database systems used in the project.

5.2 Foodies and Chefs Generation

While recipe data were collected through web scraping, user and chef profiles were generated programmatically through Python scripts. In particular, the *Faker* library was used to generate realistic synthetic data for user and chef profiles, such as usernames, email and other personal attributes. This approach allowed us to simulate a realistic application environment with different types of users.

5.2.1 User Profile Generation

The user profiles for the platform were generated entirely through synthetic data creation to simulate a diverse and realistic user base. We utilized the *Faker* library to populate each profile with a set of personal attributes, including the first name, last name, username, email address, registration date, password, and birthday.

To facilitate the authentication process during testing, a standardized naming convention was implemented: both username and password were automatically formatted as *name.surname*.

5.2.2 Chef Profile Generation

A hybrid approach was adopted for the creation of chef profiles to ensure the presence of authentic professional identities within the system. The first and last names, were collected via web scraping from the established recipe websites named in the previous paragraphs, providing a foundation of real-world culinary personas.

Then, all additional profile information and metadata were generated using the *Faker* library, effectively blending real identity data with synthetic attributes to complete the professional profiles. Similarly to the standard user profiles, we enforced the *name.surname* format for both usernames and passwords to satisfy the same testing purposes.

5.3 Dataset Statistics

At the end of the extraction and pre-processing operations, the final data were stored into JSON files. These files contain the personal information of Foodies and Chefs, alongside the comprehensive details of all scraped recipes. In terms of volume, the final dataset size is approximate:

- Dataset of Recipes: ~ 53 MB
- Dataset of Users (both Foodies and Chefs): ~ 2.5 MB

It is crucial to underline that recipes must be linked to their authoring Chefs, and simultaneously associated with the Foodies who save them as favorites. However, the technical implementation and storage of these relational mapping logic depend heavily on the chosen database architecture, which will be detailed in depth in Chapter 6.

Chapter 6

DB Design

Given the large volume of data the system must handle, the choice of database architecture is crucial to ensuring high performance and service reliability. To achieve optimal efficiency and satisfy the specific functional needs of the application, we decided to distribute our data across three different types of large-scale database, each chosen for its specific strengths: **MongoDB**, **Neo4j**, and **Redis**.

6.1 Volume consideration

The architectural decisions behind our database design are based on specific considerations regarding the volume of daily active users and their interactions with the application.

The platform currently registers the following entities:

Entity	Count
Foodies	5.000
Chefs	93
Recipes	18.832

Table 6.1: Platform registered entities

Additionally, we estimate a pool of about 20,000 unregistered guest users. Assuming a daily active user rate of 25% for regular users and 20% for Chefs, we estimate the daily traffic shown in the following table.

User Type	Active Users
Active Foodies	~1,250
Active Guests	~5,000
Active Chefs	~20

Table 6.2: Platform registered entities

As far as the operations are concerned, we have divided them into three main groups based on the frequency of their execution, as detailed in Tables 6.3, 6.4, and 6.5.

Operation	Description	Est. Volume
<i>Homepage Load (Recent Recipes)</i>	Performed once per user upon login	≈ 6.250 queries/day
<i>Viewing Recipe Details</i>	Estimated at 5 views per user, representing the most intensive operation	≈ 31.250 queries/day
<i>Search by Title</i>	Approximately 2 searches per user on average	≈ 12.500 queries/day
<i>Browsing Saved Recipes</i>	Estimated at 1.5 views per Foodie. This operation frequently involves sorting (e.g., by saving date) and filtering (by category or difficulty)	$\approx 1.5 \times 1.250 =$ 1.875 queries/day
<i>Search by Category</i>	Performed by about 70% of the daily users	$\approx 0,7 \times 6.250 =$ 4.375 queries/day
<i>Foodie/Chef login</i>	Estimated at one login session per active registered user (Foodies and Chefs) per day	$\approx 1.250 + 20 =$ 1.270 queries/day

Table 6.3: High-Frequency Operations

Operation	Description	Est. Volume
<i>Chef Profile Search</i>	Consulted by only 10% of users	$\approx 0,1 \times 6.250 = 625$ queries/day
<i>Smart Fridge Suggestions</i>	Accessed on average once a day per Foodie	≈ 1.250 queries/day
<i>Smart Fridge Update</i>	Since the fridge is updated every 2-3 days, we estimate roughly 500 write operations/day	≈ 500 writes/day
<i>Shopping List</i>	About 50% of Foodies modify their list daily	≈ 625 queries/day
<i>Saving Recipes (Favorites)</i>	Assuming 30% of Foodies save a recipe daily, we have approximately 375 updates/day on MongoDB	≈ 375 updates/day
<i>Creating New Recipes</i>	Considering the publication of about 1 recipe per week per active Chef (roughly 15% daily across 20 Chefs), we estimate the insertion of new recipes/day awaiting Admin approval	≈ 3 insertions/day
<i>Admin approval/Discard of a recipe</i>	Directly correlated to the creation rate of new recipes. The Admin is expected to process the pending queue daily, resulting in a low but steady volume of write operations	≈ 3 operations/day

Table 6.4: Medium-Frequency Operations

Note: Although the absolute volume for creating and approving recipes is currently low, these operations are classified as medium-frequency with scalability in mind, anticipating a future increase in the volume of Chefs (currently limited to 93). In fact, projecting an application growth to 500-1000 professionals, the volume of these operations would proportionally rise to an estimated 75-150 queries per day.

Operation	Description	Est. Volume
<i>Removing a Favorite</i>	Estimated at 1 removal every two weeks per logged-in user, generating a minimal impact	≈ 89 queries/day
<i>Foodie/Chef registration</i>	Representing the organic growth of the platform, we estimate a daily acquisition rate of roughly 1% of the active registered user base, predominantly comprising new Foodies	≈ 15 operations/day
<i>Recipe Deletion</i>	A recipe deletion (1 every month) is considered to be a low frequent operation, therefore it had a lower impact on the application	≈ 3 operation/day
<i>Admin approval/Discard of a chef</i>	Following a new Chef registration request, the Admin must manually validate their credentials. Given the specialized and restricted nature of this role, this occurs very sporadically	≈ 1 operation/day
<i>Account Deletion</i>	Deleting a Foodie profile (1 every three months) has a workload impact close to zero. The deletion of Chef profiles is considered statistically null	≈ 0

Table 6.5: Low-Frequency Operations

Aggregating the estimates above, the system manages over 56,250 read operations against merely 1,900 daily write operations, highlighting a traffic composition that is 96.7% read-only. Given this heavily read-centric workload and in accordance with the non-functional requirements for high availability, the entire database architecture has been designed to maximize read performance. Consequently, the design prioritizes data retrieval speed, deliberately accepting a trade-off that may result in slightly higher latency for write operations.

6.2 Document DB - MongoDB

MongoDB was adopted as the primary database for storing the application’s core entities: Chefs, Foodies and Recipes. This architectural choice is strongly driven by its schema-less nature: such structural flexibility is crucial for modeling and storing highly heterogeneous and complex datasets.

To maximize system performance, redundant fields were introduced, allowing us to bypass complex queries and reduce read latency. Finally, the implementation of a robust indexing strategy guarantees optimal response times, ensuring high scalability and the capability to efficiently handle massive data volumes.

6.2.1 Collections

The MongoDB database is structured into the following collections

- Foodie
- Chef
- Recipes

Foodie

```

1 {
2   "_id": {
3     "$oid": "69cea20c14f896db429b5a79"
4   },
5   "username": "sierra.mitchell",
6   "name": "Sierra",
7   "surname": "Mitchell",
8   "email": "sierra.mitchell@hotmail.com",
9   "registration_date": {
10     "$date": "2024-11-12T00:00:00.000Z"
11   },
12   "password": "$2b$04$fJtPdwoGxDMSbOnsEE6RkeJX9gTvxd7AH322Bhh2dg1.Zb8psmvFi",
13   "saved_recipes": [
14     {
15       "id": "69cea24d14f896db429b9b33",
16       "title": "Gluten-Free Waffles - Katy Beskow",
17       "category": "Egg-free",
18       "difficulty": "Very easy",
19       "image_url": "https://www.allrecipes.com/a65db1500608e88f.jpg",
20       "saving_date": {
21         "$date": "2026-04-01T22:00:00.000Z"
22       },
23       "chef": {
24         "id": "69ce825a14f896db429a14a6",
25         "name": "Marco",
26         "surname": "Pierre White"
27       }
28     },
29     {...}
30   ],
31   "birthdate": {
32     "$date": "2006-07-25T00:00:00"
33   }
34 }

```

Within the Foodie collection, alongside the user's personal profile and registration details, each document includes an array of partial-embedded documents called `saved_recipes`. For each saved recipe, the document retains the ID, title, category, difficulty, image URL, saving date, and essential details about the chef (ID, first name, and last name).

The rationale behind this data modeling choice is that, as discussed in Section 6.1, accessing saved favorites constitutes one of the application’s most frequent read operations. By applying the partial embedding pattern, all the required fields to render the recipe preview, filter by category/difficulty, and sort by saving date are immediately available within the parent document.

Although embedding data increases the document’s dimension, the risk of unbounded growth is inherently mitigated in this use case. Statistics from real-world recipe platforms indicate that users rarely exceed 150 saved recipes. For our dataset generation, we adopted a conservative approach, randomly assigning between 0 and 200 saved recipes per user. Even in the worst-case scenario with 200 embedded items, the total size of the Foodie document remains orders of magnitude below MongoDB’s strict 16MB physical limit.

Chef

```
1 {
2   "_id": {
3     "$oid": "69ce825a14f896db429a146f"
4   },
5   "name": "Nadiya",
6   "surname": "Hussain",
7   "email": "nadiya.hussain@yahoo.com",
8   "password": "$2b$04$178iTz6zwQsFYR7r1eSuIu0rEqJZx9man0jMx7cR5lUzkfsGE.X/a",
9   "birthdate": {
10     "$date": "1949-05-18T23:00:00.00"
11   },
12   "username": "nadiya.hussain",
13   "new_recipes": [
14     {
15       "id": "69cea24c14f896db429b7422",
16       "title": "Vegetable vegan biriyani with carrot salad",
17       "image_url": "https://vegetable-vegan-biriyani-with-carrot-salad.jpeg",
18       "creation_date": {
19         "$date": "2026-02-20T23:53:00"
20       },
21       "num_saves": 32
22     },
23     {...}
24   ],
25   "old_recipes": [
26     "69cea24c14f896db429b7b6a",
27     "69cea24c14f896db429b81a9",
28     "69cea24e14f896db429ba682",
29     ...
30   ],
31   "popular_recipes": [
32     {
```



```

33     "id": "69cea24e14f896db429b9e89",
34     "title": "Grilled Tandoori Chicken Thighs",
35     "image_url": "https://grilled-chicken-thighs.png",
36     "creation_date": {
37         "$date": "2022-09-29T12:26:02"
38     },
39     "num_saves": 45
40 },
41 {...}
42 ],
43 "recipes_to_confirm": [
44     {
45         "id": "6a2180330fc2e5349e11dc84",
46         "title": "Sweet Cherry Pie",
47         "image_url": "https://sweet-cherry-pie.png",
48         "creation_date": {
49             "$date": "2026-06-04T13:40:03"
50         }
51     }
52 ],
53 "tot_saves": 5206,
54 "tot_recipes": 162,
55 "status": "APPROVED"
56 }

```

Within the Chef collection, each document contains the chef's personal information (such as name, surname, username, email, password, and birthdate), along with a strategic combination of partial embedding, document linking (references), and computed fields to optimize read operations.

Specifically, the `new_recipes` array holds the chef's 15 most recently written and approved recipes, sorted in descending order by creation date. For each of these recipes, the document embeds only the fields necessary to render the preview card in the chef's personal area. Since the view is paginated, displaying 5 recipes per page (embedding the 15 most recent items), the first three pages are immediately ready to be served to the user, who is statistically most likely to access them.

For older recipes (those accessed from the fourth page onward), the system employs document linking to prevent unbounded document growth. The `old_recipes` array stores only the recipe IDs. When an older recipe needs to be visualized, the application performs a subsequent query on the Recipes collection using these IDs. This operation remains highly efficient due to the primary index defined on the `_id` field.

A similar partial embedding strategy is applied to the `popular_recipes` array, which contains previews of the chef's most successful creations. Considering that the average number of saves per recipe on the platform ranges between 20 and 30, we

defined a recipe as "popular" when it exceeds 40 saves. By definition, this subset remains small, making embedding a safe and highly performant choice for rapid retrieval.

Additionally, the `recipes_to_confirm` array closely follows this architectural approach by employing the partial embedding pattern for recipes currently awaiting admin approval. Instead of embedding the entire recipe entity, it stores only the preview details. This choice is justified under the assumption that the turnaround time for admin review is relatively short; therefore, the volume of pending items at any given time will remain naturally bounded and will never compromise the document's dimension.

The document also features a status field to manage the onboarding workflow. When a chef submits a registration request, this is set to "PENDING". In this state, the document is essentially immutable: the user cannot modify personal information, create new recipes, or access the personal dashboard. Once the admin evaluates and approves the request, the status transitions to "APPROVED", granting the chef full access to the application.

Finally, the document introduces controlled redundancy through the `tot_recipes` and `tot_saves` fields. Although this design choice requires additional update operations to maintain data consistency across collections, storing these pre-computed totals drastically simplifies and accelerates the statistical analysis features required by the admin dashboard, such as the execution of the Bayesian Ranking algorithm.

Admin

```
1 {
2   "_id": {
3     "$oid": "69cea60d14f896db429babe3"
4   },
5   "password": "$2a$10$ZWl8yOftWkI36aJQbvLHh.b.BqYYEVP6tvHHfgmBzMiSbGma1jWC",
6   "username": "admin",
7   "recipes_to_approve": [
8     {
9       "chef": {
10         "id": "69ce825a14f896db429a146f",
11         "name": "Nadiya",
12         "surname": "Hussain"
13       },
14       "id": "6a2180330fc2e5349e11dc84",
15       "title": "Sweet Cherry Pie",
16       "image_url": "https://sweet-cherry-pie.png",
17       "creation_date": {
18         "$date": "2026-06-04T13:40:03.537Z"
19       }
20     }
21   ]
22 }
```

```

21 ],
22 "chefs_to_approve": [
23   {
24     "_id": {
25       "$oid": "6a21a800f8321012c53e1d2f"
26     },
27     "username": "andrea.ricci",
28     "name": "Andrea",
29     "surname": "Ricci",
30   }
31 ]
32 }

```

Although the Admin represents a distinct role with elevated system privileges, its document is housed within the Chef collection. This design choice avoids the unnecessary overhead of creating and maintaining a dedicated collection for a single element. The Admin document is structurally minimalist, containing only a username and a hashed password, omitting personal details that are irrelevant to this operational role.

To efficiently power the administrative dashboard, the document utilizes the partial embedding pattern for its two primary task queues: `recipes_to_approve` and `chefs_to_approve`.

Instead of relying on costly, real-time queries across multiple collections to find pending items, the Admin document directly embeds an array of preview objects. The `recipes_to_approve` array contains only the essential fields, such as the recipe ID, title, image URL, creation date, and brief chef details, required to render the pending recipes interface. Similarly, the `chefs_to_approve` array embeds a lightweight preview of the registration requests (ID, full name, username).

This strategy ensures that when the Admin accesses the dashboard, a single document retrieval provides all the data necessary to populate the moderation views. Furthermore, the risk of unbounded document growth is naturally mitigated by the workflow semantics: these arrays act as transient queues. As the Admin approves or rejects requests, the corresponding embedded objects are removed from the arrays, keeping the document size strictly bounded, lightweight, and highly performant.

Recipe

```

1 {
2   "title": "Pumpkin Roll",
3   "image_url": "https://www.pumpkin-roll.jpg",
4   "category": "Desserts",
5   "difficulty": "Easy",
6   "prep_time": "20 mins",
7   "presentation": "This pumpkin roll recipe makes a moist, spiced pumpkin
   cake with a vanilla cream cheese filling, rolled up and dusted with

```

```

8     powdered sugar for a standout dessert that always impresses. It's
9     easier to make than it looks and perfect for Thanksgiving or Christmas.",
10    "ingredients": [
11      { "name": "white sugar", "quantity": "1 cup" },
12      { "name": "pumpkin puree", "quantity": "2/3 cup" },
13      { "name": "eggs", "quantity": "3 large" },
14      { "name": "cinnamon", "quantity": "1/2 teaspoon" },
15      { "name": "flour", "quantity": "3/4 cup" },
16      { "name": "baking soda", "quantity": "1 teaspoon" },
17      { "name": "cream", "quantity": "8 ounces" },
18      { "name": "icing sugar", "quantity": "1 cup" },
19      { "name": "butter", "quantity": "2 tablespoons" },
20      { "name": "vanilla extract", "quantity": "1/4 teaspoon" }
21    ],
22    "preparation": "Preheat oven to 375 F (190 C). Grease a 10x15-inch pan.
23    Mix sugar, pumpkin, eggs, and cinnamon. Add flour and baking soda. Bake
24    15-20 minutes. Roll while warm. Prepare filling with cream cheese, sugar,
25    butter, vanilla. Unroll, spread filling, roll again, refrigerate, and
26    serve.",
27    "chef": {
28      "id": "69a070ce6758c1dfc03e05f2",
29      "name": "Melissa",
30      "surname": "Hemsley"
31    },
32    "creation_date": "2022-01-03T20:17:25",
33    "num_saves": 36,
34    "status": "APPROVED"
35  }

```

Within the Recipes collection, each document represents a complete and autonomous entity that exhaustively models all information related to a recipe. Beyond the synthetic attributes used to render the preview cards (such as title, image URL, etc.), the extended document stores the complete operational details, including a narrative description (**presentation**), a structured array of ingredients (containing names and quantities), and the textual instructions for preparation. Regarding the chef, this information is stored as an embedded document reporting the ID, name, and surname: in this way, we are able to include the chef's name in the recipe details and make it a clickable link through which the specific chef's recipes can be displayed.

In addition, each document includes a status field: upon submission by a chef, the recipe is created with a "PENDING" status. During this phase, the document remains invisible to the public: it cannot be searched, viewed, or saved by Foodies. Only a positive review by the Admin changes the status to "APPROVED", making the recipe officially public within the platform.

An important note concerns the `num_saves` field, which acts as a redundant denormalized field in the database. Despite this redundancy, it is crucial whenever a user saves

or removes a recipe from their favorites. These actions directly affect the recipe's popularity status and, consequently, its inclusion in the chef's `popular_recipes` list. Without this pre-calculated counter, the system would be forced to execute costly cross-collection queries to compute the total saves on every interaction. Therefore, while it is true that it is a redundant field and this adds complexity when adding a new favorite since the counters need to be updated, it makes the operation of maintaining the chef's popular recipes much simpler.

6.3 Key-Value DB - Redis

The adoption of a Key-Value database within our architecture was driven by its distinct performance characteristics, making it the ideal solution for managing simple, high-frequency operations. Specifically, it is perfectly suited for the rapid and continuous updating of the ingredient lists associated with the Smart Fridge and Shopping List features.

Furthermore, this data store serves a crucial dual purpose within the system: first, it acts as a fast, in-memory dictionary to store and validate the unique allowed ingredients across the application; second, it functions as a highly efficient caching layer. By temporarily storing the personalized recipe recommendations generated by the graph database, it significantly reduces query latency.

6.3.1 Structure

Data within Redis is classified into four separate sets of keys, structured according to the patterns described in the following paragraphs.

```
1 KEYS *
2     "Allowed_ingredients"
3     "Foodie:sierra.mitchell:shoppingList"
4     "Foodie:sierra.mitchell:smartFridge:suggestions"
5     "Foodie:sierra.mitchell:smartFridge:ingredients"
```

Allowed Ingredients

The Allowed Ingredients are a centralized in-memory set used to quickly validate ingredients across the entire platform. It ensures strict data consistency not only when Foodies add items to their Shopping List or Smart Fridge, but also by guiding Chefs during the recipe creation process, ensuring that only officially permitted ingredients enter the system's database.

Key: `Allowed_ingredients`

Value: A set of unique allowed ingredients

```
1 SMEMBERS Allowed_ingredients
2     "peel"
3     "pork belly"
4     "mascarpone cheese"
5     "instant gluten free yeast"
6     "brown sugar"
7     "apple juice"
8     "dried berries"
9     "cauliflower rice"
10    "chili oil"
11    "bechamel sauce"
12    "pizza sauce"
13    "mushroom powder"
14    "falafel"
15    "protein powder"
16    "pastry cream"
17    "..."
```

Shopping List

The Shopping List is a user-specific set where Foodies can store ingredients they intend to purchase. To guarantee data uniformity, all inputs are automatically validated and normalized into a lowercase format before storage, regardless of the user's original casing.

Key: Foodie:<foodie_username>:shoppingList

Value: A set of unique ingredients saved by a foodie for future purchase

```
1 SMEMBERS Foodie:sierra.mitchell:shoppingList
2     "butter"
3     "bread"
4     "chocolate"
5     "spaghetti pasta"
6     "avocados"
```

Smart Fridge

The Smart Fridge is a component that allows users to manage the ingredients available in their pantry and receive suggestions about recipes that can be made using at least 3 of those ingredients. It applies the exact same lowercase normalization and validation policy as the Shopping List.

According to what we have just said, we use two distinct types of keys to track user ingredients and cache the results of the recommended recipes.

Key: Foodie:<foodie_username>:smartFridge:ingredients

Value: A set of unique ingredients that the foodie currently has in its pantry

```
1 SMEMBERS Foodie:sierra.mitchell:smartFridge:ingredients
2     "spaghetti pasta"
3     "butter"
4     "avocados"
```

Key: Foodie:<foodie_username>:smartFridge:suggestions

Value: A cached JSON string representing a sorted list of recipe recommendations, generated based on the foodie's available ingredients

```
1 GET Foodie:sierra.mitchell:smartFridge:suggestions
2     [ {
3         "id" : "69cea24e14f896db429ba7b7",
4         "title" : "Best Chicken Panini",
5         "imageURL" : "https://www.best-chicken-panini.jpeg",
6         "chef" : "Gordon Ramsay",
7         "matchCount" : 3,
8         "matchedIngredients" : [ "bread", "butter", "avocados" ],
9         "chef_id" : "69ce825a14f896db429a14a5"
10    },
11    {
12        "id" : "69cea24e14f896db429ba6f9",
13        "title" : "Avocado Breakfast Toast",
14        "imageURL" : "https://www.avocado-breakfast-toast.png",
15        "chef" : "Dr Rupy Aujla",
16        "matchCount" : 3,
17        "matchedIngredients" : [ "bread", "butter", "avocados" ],
18        "chef_id" : "69ce825a14f896db429a1473"
19    },
20    { ... }
21 ]
```

The recommendations are initially computed by querying the graph database and subsequently cached in Redis to drastically reduce read latency. To maximize performance, the system employs a highly optimized cache management strategy: if

a new ingredient is added, the cache is cleared; however, if an ingredient is removed, the cached suggestions are updated in-place by dynamically recalculating the match counts, avoiding a full cache invalidation.

Furthermore, if a recommended recipe is deleted from the platform, the system adopts a "lazy update" approach: the obsolete recipe remains in the cache and is removed only upon a user's read attempt. In this way, we eliminate the need for heavy synchronization between MongoDB and Redis.

6.3.2 Keys TTL

Building upon the lazy update strategy described in the previous section, Redis is intentionally not updated synchronously during major deletion events. This includes the deletion of a Chef's profile (along with all their associated recipes), the removal of a specific recipe, or the deletion of a Foodie's profile.

To guarantee eventual data consistency without introducing heavy synchronization overhead, a Time-To-Live (TTL) of 15 days is applied to all user-specific keys. The only exception to this policy is the `Allowed_ingredients` key, which serves as a persistent system dictionary and therefore never expires.

Furthermore, whenever an operation modifies a key, its TTL is automatically reset, reflecting the fact that the user is actively utilizing the service. Consequently, if a Foodie permanently deletes their account, the system safely relies on this expiration mechanism to automatically clean up their Shopping List and Smart Fridge keys after a maximum of 15 days.

6.4 Graph DB - Neo4j

The integration of a Graph Database within our architecture was driven by its inherent nature: it allows us to explicitly define and seamlessly navigate the complex relationships between recipes, the ingredients used to make them, and the chefs who created them.

This graph structure enables us to offer a comprehensive and highly interconnected experience to our Foodies. Specifically, it powers the logic behind the Smart Fridge: starting from a list of inputted ingredients, the system traverses the graph to find recipes that can be made using at least 3 of those ingredients, ordering the results by the number of matched items. Furthermore, the graph engine allows us to suggest similar chefs (based on the ingredients they most frequently use), recommend similar recipes (based on shared ingredients), and even suggest complementary ingredients to purchase alongside those already present in the user's Shopping List.

6.4.1 Graph Nodes

To support these features, the following entities have been modeled as nodes within the graph database:

- **Chef**: represents the authors of the recipes. Each node stores its unique identifier along with basic information retrieved from MongoDB, specifically the `mongo_id`, `name`, and `surname`.
- **Ingredient**: represents the ingredients used in recipes. Each ingredient is modeled as an independent node identified by a unique identifier and its name.
- **Recipe**: represents the recipes in the system. Each node contains its unique identifier and a lightweight subset of information retrieved from MongoDB, such as the `mongo_id`, `title`, `category` and `image URL`, which are the essential elements required to quickly render recipe previews across the application.

6.4.2 Graph Relationships

As shown in Figure 6.1, the relationships established within the graph database are the following:

- **:USED_IN**: connects an *Ingredient* to a *Recipe*, representing which recipes utilize that specific ingredient.

`(:Ingredient)-[:USED_IN]->(:Recipe)`

- **:USES**: connects a *Recipe* to an *Ingredient*, representing which ingredients are required for a given recipe. This relationship acts as the inverse navigation of `:USED_IN`, enabling logically linear and efficient queries when starting the traversal from a recipe node.

`(:Recipe)-[:USES]->(:Ingredient)`

- **:WRITTEN_BY**: connects a *Recipe* to a *Chef*, representing the author of the recipe.

`(:Recipe)-[:WRITTEN_BY]->(:Chef)`

- **:WROTE**: connects a *Chef* to a *Recipe*. This relationship acts as the inverse navigation of `:WRITTEN_BY`, allowing for straightforward query paths when starting from the chef node.

`(:Chef)-[:WROTE]->(:Recipe)`

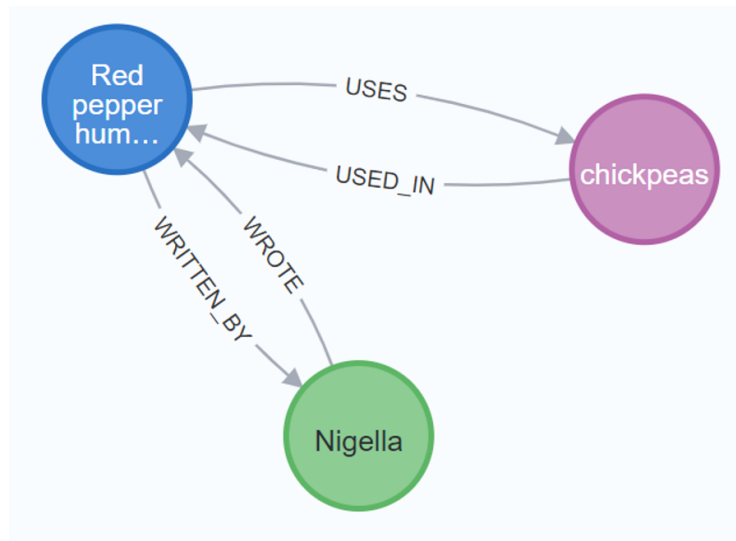


Figure 6.1: Single node relationship

These mirrored relationships (one being the exact inverse of the other) were deliberately introduced to enforce logical clarity within the architectural model. This design emphasizes that the starting node of a query dictates a unidirectional flow, distinguishing an efficient, targeted traversal from a costly full-graph scan. At the same time, it is acknowledged that modern versions of the Cypher engine are fully capable of traversing a single relationship in either direction without incurring any additional computational costs or unnecessary node visits.

Graph example

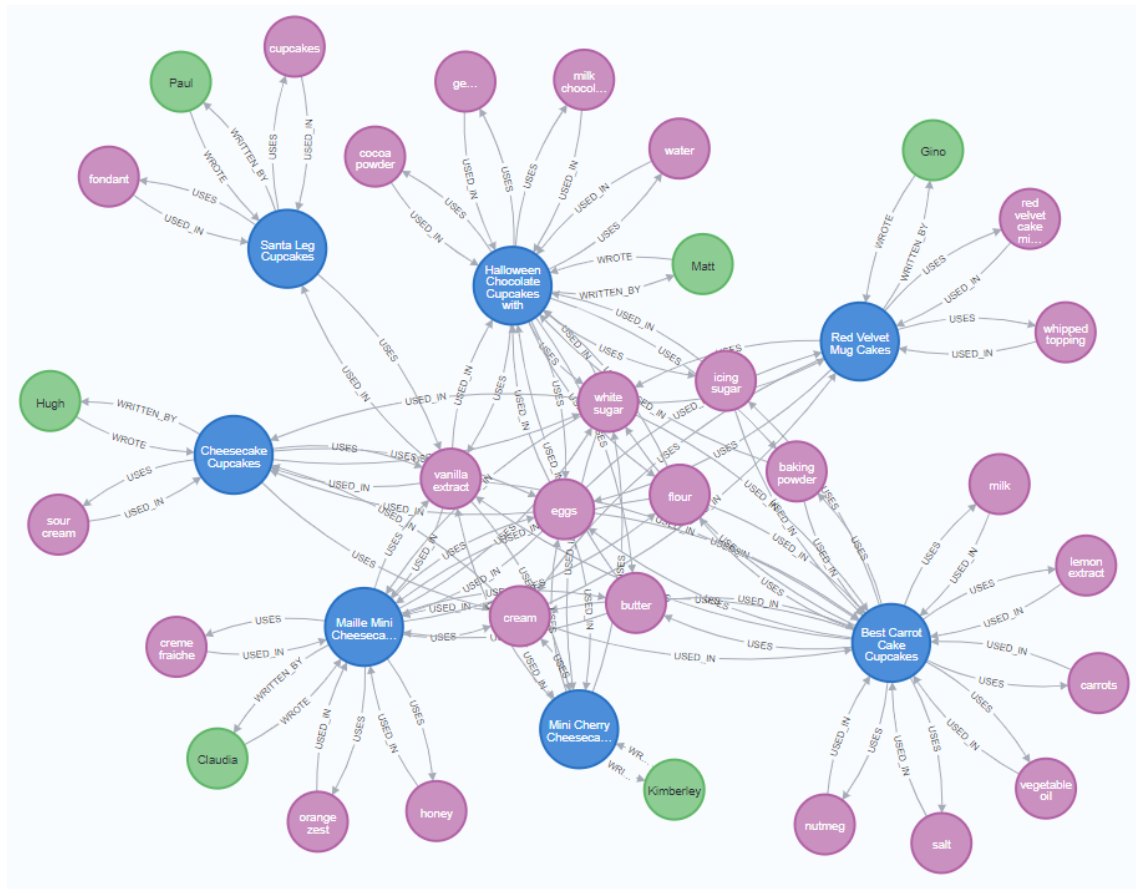


Figure 6.2: Example of the nodes and relationships

Chapter 7

Implementation

In this chapter, the details of the application's implementation will be discussed thoroughly.

The Spring Boot framework was used to implement the application in order to build a robust and scalable project. The goal of this section is to discuss and explain the choices regarding our application and the specific role of the components.

The application follows the Spring Boot MVC (Model View Controller) architectural pattern. To ensure a clean separation of concerns, the codebase is systematically organized into distinct packages, each dedicated to a specific role identified during the development phase.

7.1 Spring

The project was developed using the Spring Boot framework, as it is widely adopted and is a robust solution that ensures scalability and provides access to the most recent technologies and dependencies.

Indeed, it simplifies dependency management, provides an embedded server for testing the application, and facilitates the development process. Moreover, Spring Boot includes a variety of useful tools and technologies; for this reason, we were also able to create and test REST APIs effectively.

7.2 Security

In this application, a security configuration was implemented to ensure appropriate protection mechanisms and to enhance the overall robustness of the system.

In particular, the chosen authentication system supports login, authorization, and role-based access control using JWT (JSON Web Tokens). This approach enables secure authentication without the need for server-side sessions, reducing potential security vulnerabilities and improving efficiency in data handling. Furthermore, it protects the various endpoints of the application, ensuring that only users with the appropriate roles can access specific resources.

The method adopted to implement the security configuration is illustrated as follows:

```

1  /**
2   * Configures security rules and filter chain.
3   * @param http HttpSecurity object used to configure security
4   * @return configured SecurityFilterChain
5   * @throws Exception if configuration fails
6   */
7  @Bean
8  public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
9
10     http
11         //disable CSRF since the application uses JWT (stateless)
12         .csrf(csrf -> csrf.disable())
13
14         //no server-side session: authentication is token-based
15         .sessionManagement(session ->
16             session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
17
18         //define endpoint authorization rules
19         .authorizeHttpRequests(auth -> auth
20
21             // PUBLIC endpoints
22             .requestMatchers("/api/auth/**").permitAll()
23             .requestMatchers(HttpMethod.GET, "/api/recipes/**").permitAll()
24             .requestMatchers(HttpMethod.GET,
25                 "api/ingredients/**").permitAll()
26             .requestMatchers("/error").permitAll()
27
28             // SWAGGER endpoints
29             .requestMatchers("/v3/api-docs/**", "/swagger-ui/**",
30                 "/swagger-ui.html").permitAll()
31
32             // CHEF - recipe management
33             .requestMatchers(HttpMethod.POST, "/api/recipes/**")
34             .hasRole("CHEF")
35             .requestMatchers(HttpMethod.PUT, "/api/recipes/**")
36             .hasRole("CHEF")
37             .requestMatchers(HttpMethod.DELETE, "/api/recipes/**")
38             .hasRole("CHEF")
39
40             // CHEF - profile management
41             .requestMatchers("/api/chefs/**").hasRole("CHEF")
42
43             // FOODIE - profile management
44             .requestMatchers("/api/foodies/**").hasRole("FOODIE")
45
46             // FOODIE - saved recipes
47             .requestMatchers("/api/savedrecipes/**").hasRole("FOODIE")
48
49             // FOODIE - SmartFridge

```

```

49         .requestMatchers("/api/fridge/**").hasRole("FOODIE")
50
51         // FOODIE - SmartShoppingList
52         .requestMatchers("/api/shopping/**").hasRole("FOODIE")
53
54         // ADMIN endpoints
55         .requestMatchers("/api/admin/**").hasRole("ADMIN")
56
57         // All other requests require authentication
58         .anyRequest().authenticated()
59     )
60
61     //register authentication provider
62     .authenticationProvider(authenticationProvider());
63
64     // Add the JWT filter to the Spring Security filter chain. It's executed
65     // before the default UsernamePasswordAuthenticationFilter.
66     http.addFilterBefore(authenticationJwtTokenFilter(),
67         UsernamePasswordAuthenticationFilter.class);
68
69     return http.build();
70 }

```

7.3 Config

This package contains the configuration classes for the application. Specifically, the following classes are implemented:

- **GlobalExceptionHandler:** This class centralizes exception management across the application. It ensures consistent error handling and provides standardized server responses for various exception scenarios.
- **RedisConfig:** This class configures a Redis Master-Slave architecture using Sentinel, providing high availability and managing the connection pool for the application.
- **SwaggerConfig:** This class is used for the Swagger configuration, providing comprehensive documentation and a standardized interface for exploring the application's APIs.

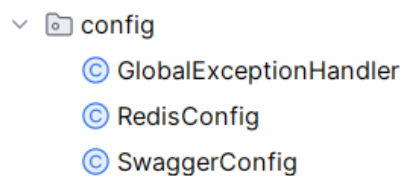


Figure 7.1: Config package folder

7.4 Event

This package contains the classes responsible for defining and structuring asynchronous tasks within the system. These classes act as data containers and identifiers for operations that are delegated to the background manager, ensuring that the main application performance is not degraded.

Task.java

This class provides an enumeration of the different task types that can be executed within the application. It defines various background operations, such as updating counters when a foodie deletes its profile or modifies its favorite recipes, as well as managing the synchronization of the Neo4j graph database when recipes or chef profiles are created or deleted.

TaskToDo.java

This class acts as a task manager, representing a specific unit of work that needs to be executed. It encapsulates the task type alongside its required data context, offering multiple overloaded constructors to handle different task payloads. Depending on the specific operation, a task can be instantiated with simple recipe and chef IDs, bulk deletion information, recipe summaries, or specific Data Transfer Objects (DTOs) tailored for graph database operations.

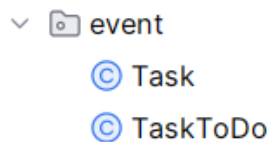


Figure 7.2: Event package folder

7.5 Model

7.5.1 MongoDB Models

Ingredients

```
1 /**
2  * Represents an ingredient inside a recipe.
3  */
4 @Getter
5 @Setter
6 @AllArgsConstructor
7 @NoArgsConstructor
8 public class RecipeIngredient {
9
```

```

10     private String name;
11     private String quantity;
12 }

```

Recipes

```

1  /**
2  * Core Recipe entity stored within the "recipe" MongoDB collection.
3  */
4  @Getter
5  @Setter
6  @NoArgsConstructor
7  @AllArgsConstructor
8
9  @Document(collection = "recipes")
10 public class RecipeMongo {
11
12     @Id
13     private String id;
14
15     @TextIndexed
16     private String title;
17
18     @Field("presentation")
19     private String presentation;
20
21     @Indexed
22     private String category;
23
24     @Field("prep_time")
25     private String prepTime;
26
27     private String preparation;
28
29     private String difficulty;
30
31     @Field("image_url")
32     private String imageURL;
33
34     private List<RecipeIngredient> ingredients;
35
36     private ReducedChef chef;
37
38     @Indexed(direction = IndexDirection.DESCENDING)
39     @Field("creation_date")
40     private LocalDateTime creationDate;
41
42     @Field("num_saves")

```



```

43     private Integer numSaves;
44
45     private String status;
46 }

```

```

1  /**
2  * Represents a lightweight, embedded sub-document stored within the Chef
3  * entity. It tracks the essential details of a recipe that has been
4  * submitted by the chef and is currently awaiting review and approval by
5  * the admin.
6  */
7  @Getter
8  @Setter
9  @AllArgsConstructor
10 @NoArgsConstructor
11 public class PendingRecipe {
12     @Field("id")
13     String id;
14     private String title;
15
16     @Field("image_url")
17     private String imageURL;
18
19     @Field("creation_date")
20     private LocalDateTime creationDate;
21
22 }

```

```

1  /**
2  * Represents a lightweight, embedded sub-document stored within the Admin
3  * entity. It tracks the essential details of a recipe (along with
4  * its author) that has been submitted by a chef and is currently awaiting
5  * review and approval by the admin.
6  */
7  @Getter
8  @Setter
9  @AllArgsConstructor
10 @NoArgsConstructor
11 public class AdminPendingRecipe extends PendingRecipe {
12
13     private ReducedChef chef;
14
15 }

```

```

1  /**
2  * Represents a lightweight, embedded sub-document stored within the Chef
3  * entity. This class implements the "partial embedding" pattern.
4  */
5  @Getter

```

```

6 @Setter
7 @NoArgsConstructor
8 @AllArgsConstructor
9 public class ChefRecipeSummary {
10
11     @Field("id")
12     private String id;
13
14     private String title;
15
16     @Field("image_url")
17     private String imageURL;
18
19     @Field("creation_date")
20     private LocalDateTime creationDate;
21
22     @Field("num_saves")
23     private Integer numSaves;
24 }

```

```

1 /**
2  * Represents a lightweight, embedded sub-document stored within the Foodie
3  * entity. This class implements the "partial embedding" pattern.
4  */
5 @Getter
6 @Setter
7 @NoArgsConstructor
8 @AllArgsConstructor
9 public class FoodieRecipeSummary {
10
11     @Field("id")
12     private String id;
13
14     private String title;
15
16     private String category;
17
18     private String difficulty;
19
20     @Field("image_url")
21     private String imageURL;
22
23     @Field("saving_date")
24     private LocalDate savingDate;
25
26     @Field("chef")
27     private ReducedChef chef;
28 }

```

Users

```
1  /**
2  * Base class representing a registered user in the system with all common
3  * attributes
4  */
5  @Getter
6  @Setter
7  @NoArgsConstructor
8  @AllArgsConstructor
9  public abstract class RegisteredUser {
10
11     @Id
12     private String id;
13
14     @NotBlank(message = "First name is required")
15     private String name;
16
17     @NotBlank(message = "Last name is required")
18     private String surname;
19
20     @NotBlank(message = "E-mail is required")
21     @Email
22     @Size(max = 50)
23     private String email;
24
25     @NotBlank
26     @Size(min = 8, max = 20)
27     private String password;
28
29     @Past(message = "Birthdate must be in the past")
30     private LocalDate birthdate;
31 }
```

```
1  /**
2  * Represents the core Chef entity stored within the "chefs" MongoDB
3  * collection.
4  */
5  @Getter
6  @Setter
7  @NoArgsConstructor
8  @AllArgsConstructor
9  @Document(collection = "chefs")
10
11 public class Chef extends RegisteredUser {
12
13     @Indexed(unique = true)
14     @NotBlank(message = "Username is required")
15     @Size(max = 20)
```

```
16     private String username;
17
18     @Field("new_recipes")
19     private List<ChefRecipeSummary> newRecipes;
20
21     @Field("old_recipes")
22     private List<String> oldRecipes;
23
24     @Field("recipes_to_confirm")
25     private List<PendingRecipe> recipesToConfirm;
26
27     @Field("popular_recipes")
28     private List<ChefRecipeSummary> popularRecipes;
29
30     @Field("tot_saves")
31     private Integer totalSaves;
32
33     @Field("tot_recipes")
34     private Integer totalRecipes;
35
36     private String status;
37 }
```

```

1  /**
2  * Represents the Admin entity stored within the "chefs" MongoDB collection.
3  */
4  @Getter
5  @Setter
6  @AllArgsConstructor
7  @NoArgsConstructor
8
9  @Document(collection = "chefs")
10 public class Admin {
11
12     @Id
13     private String id;
14
15     private String password;
16
17     @Indexed(unique = true)
18     private String username;
19
20     @Field("recipes_to_approve")
21     private List<AdminPendingRecipe> recipesToApprove;
22
23     @Field("chefs_to_approve")
24     private List<PendingChef> chefsToApprove;
25
26 }

```

```

1  /**
2  * Represents the Foodie entity stored within the "foodies" MongoDB
3  * collection.
4  */
5  @Getter
6  @Setter
7  @NoArgsConstructor
8  @AllArgsConstructor
9  @Document(collection = "foodies")
10 public class Foodie extends RegisteredUser {
11
12     @NotBlank(message = "Username is required")
13     @Size(max = 20)
14     @Indexed(unique = true)
15     private String username;
16
17     @Field("registration_date")
18     private Date registrationDate;
19
20     @Field("saved_recipes")
21     private List<FoodieRecipeSummary> savedRecipes;
22

```

```
23 }
```

```
1 /**
2  * Lightweight embedded entity representing a newly registered chef currently
3   * wait for the admin to approve its registration request.
4  */
5 @Getter
6 @Setter
7 @NoArgsConstructor
8 @AllArgsConstructor
9 public class PendingChef {
10     private String id;
11     private String username;
12     private String name;
13     private String surname;
14 }
```

```
1 /**
2  * A partially embedded projection of the Chef entity, storing only essential
3   * identity fields to optimize read performance and avoid secondary database
4   * lookups.
5  */
6 @Getter
7 @Setter
8 @NoArgsConstructor
9 @AllArgsConstructor
10 public class ReducedChef {
11     @Field("id")
12     private String id;
13     private String name;
14     private String surname;
15 }
```

7.5.2 Neo4j Models

```
1 /**
2  * Represents a Recipe node in the Neo4j graph database.
3  */
4 @Getter
5 @Setter
6 @NoArgsConstructor
7 @AllArgsConstructor
8 @Node("Recipe")
9 public class RecipeNeo4j {
10 }
```

```

11     @Id @GeneratedValue(generatorClass = UUIDStringGenerator.class)
12     private Long neo4jId;
13
14     @Property("mongo_id")
15     private String mongoId;
16
17     private String title;
18     private String imageURL;
19     private String category;
20
21     private List<IngredientNeo4j> ingredients;
22     private ChefNeo4j chef;
23 }

```

```

1  /**
2  * Represents a Chef node in the Neo4j graph database.
3  */
4  @Getter
5  @Setter
6  @NoArgsConstructor
7  @AllArgsConstructor
8  @Node("Chef")
9  public class ChefNeo4j {
10
11     @Id @GeneratedValue(generatorClass = UUIDStringGenerator.class)
12     private String neo4jId;
13
14     @Property("mongo_id")
15     private String mongoId;
16
17     private String name;
18     private String surname;
19 }

```

```

1  /**
2  * Represents an Ingredient node in the Neo4j graph database.
3  */
4  @Getter
5  @Setter
6  @NoArgsConstructor
7  @AllArgsConstructor
8  @Node("Ingredient")
9  public class IngredientNeo4j {
10
11     @Id @GeneratedValue(generatorClass = UUIDStringGenerator.class)
12     private String id;
13
14     private String name;
15 }

```

7.6 Repository

The repository layer is responsible for interacting with the databases, specifically MongoDB and Neo4j. We leverage Spring Boot’s repository features to create an abstraction layer that includes standard CRUD operations, also simplifying data management. In addition, for custom queries in MongoDB and Neo4j, the `@Query` and `@Aggregation` annotations are used to handle advanced operations, ensuring both flexibility and high performance.

Within our application, we utilize basic queries for routine database management involving users, chefs, and recipes, alongside more complex queries designed for analytics and recommendations. These advanced functionalities will be discussed in further detail in Section 8.

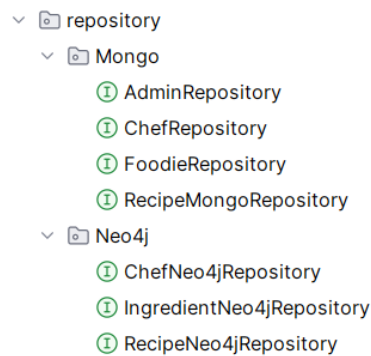


Figure 7.3: Repository packages

7.7 Service

The service layer serves as the backbone of the application, encapsulating its core business logic. It acts as a crucial intermediary between the controllers and the repositories. Furthermore, it manages transactional operations and enforces a strict separation of concerns.

Additionally, this layer is responsible for enforcing business rules, verifying data integrity, and managing errors prior to any repository interaction, thereby ensuring robust data processing and enhanced overall performance.

AdminService.java

It encapsulates the admin business logic. It handles pending recipes and chef registration requests, allowing administrators to review the full details of these submissions before deciding to approve or discard them. Upon recipe or chef’s registration approval, it orchestrates the integration of data across the MongoDB and Neo4j architectures, securely offloading graph synchronization to the background manager. Finally, it manages paginated views of pending entities and generates

comprehensive system analytics, including user growth metrics, category trend analyses, and advanced chef evaluations powered by a Bayesian ranking algorithm.

AuthService.java

This service manages the authentication in the application. It handles the registration process for both foodies and chefs. While foodie registrations are processed and persisted immediately, chef applications are intelligently routed to an administrative queue for approval. Additionally, it orchestrates the authentication process by validating user credentials and generating secure JSON Web Tokens (JWT) for authorized session management.

ChefService.java

This service encapsulates the core business logic for managing chefs and their culinary content. It allows chefs to update specific personal details, strictly limited to their email, password, and birthdate for security reasons, and handles the submission of new recipes. The service provides advanced paginated views of a chef's portfolio, allowing them to browse their recipes ordered by publication date, isolate highly popular recipes (those exceeding a specific threshold of user saves), or monitor submissions currently pending, waiting for the admin's approval.

Additionally, chefs can access the complete details of any of their recipes, whether already published or still pending, and seamlessly withdraw pending submissions if needed. To ensure optimal system responsiveness, resource-intensive operations, specifically, the deletion of a single published recipe and the complete removal of a chef's profile along with the cascading deletion of all their associated recipes, are securely offloaded to an asynchronous processing queue.

FoodieService.java

This service encapsulates the core business logic associated with Foodies. It provides essential functionalities allowing users to update their personal credentials, search for specific chefs by surname, and dynamically add or remove recipes from their favorites. The service features advanced pagination capabilities to retrieve and sort saved recipes, while also allowing foodies to inspect their full details by simply clicking on their respective previews. To enrich the user experience advanced, on-demand recommendation features are integrated: when accessing a recipe, foodies can explicitly click to reveal a dedicated section suggesting three similar recipes based on shared ingredients. Similarly, visiting a chef's profile includes a clickable section that highlights similar culinary creators grouped by their common ingredient usage.

Finally, complex operational overhead, triggered when a foodie deletes their profile or adds/removes a favorite, is securely offloaded to the asynchronous background manager. This task queue efficiently updates the `num_saves` counters across both the `Recipes` and `Chefs` collections, and dynamically re-evaluates the chef's embedded

array of popular recipes, effectively preserving the main system’s responsiveness and efficiency.

IngredientService.java

This service acts as a crucial data validation layer, ensuring consistency across all culinary inputs within the application. It uses an in-memory Redis Master-Slave architecture to efficiently verify whether a submitted ingredient exists within the system’s official allowed ingredients. By performing rapid, low-latency checks, this service actively prevents the introduction of anomalous or non-standardized data into recipes, shopping lists, and smart fridge inventories.

Furthermore, the service exposes a dedicated functionality to retrieve the complete, exhaustive list of all allowed ingredients. This feature is strategically designed to empower the client-side application: by providing the necessary dataset for guided input mechanisms (such as auto-completion or dropdown menus), it proactively steers user data entry, effectively filtering out invalid items before they even reach the backend validation layer.

LowLoadManager.java

This service acts as a strategic optimization component designed to execute resource-intensive, eventual-consistency operations without degrading main application performance. It relies on a thread-safe concurrent queue to store deferred background tasks, periodically evaluating the underlying operating system’s CPU load to determine execution safety.

When system utilization falls below a predefined threshold (fixed at 30%), it safely dispatches critical background tasks (ten at the time). Specifically, it manages the synchronization of nodes and relationships within the Neo4j graph upon a recipe’s approval, a recipe’s deletion, or the complete removal of a chef’s profile. Additionally, it handles the cascading updates of the `num_saves` counters across the `Recipes` and `Chefs` collections, which are triggered whenever a foodie modifies their favorite list or permanently deletes their account.

RecipeService.java

This service entirely manages the retrieval, searching, and filtering of the platform’s culinary content. Primarily, it strictly allows users to inspect the full details of recipes that have successfully passed the verification and approval process. To ensure a seamless and high-performing exploration of the catalog, the service implements advanced navigation capabilities strictly built upon paginated views; these allow users to filter recipes by category, browse the comprehensive portfolio of an individual chef, or perform flexible text-based searches using title substrings. Finally, the component optimizes the content discovery experience by dynamically identify and retrieve the most popular recipe within each category.

ShoppingListService.java

This service manages the user's shopping list, providing straightforward functionalities to view, add, and remove items. To ensure strict data consistency, all user inputs are automatically normalized to lowercase and validated against the system's allowed ingredients dictionary. By leveraging Redis sets for storage, the service naturally guarantees the uniqueness of the list items while enabling highly performant read and write operations.

Furthermore, to guide users during their grocery planning, the service integrates an on-demand, clickable recommendation feature: for each ingredient currently in the shopping list, it suggests three complementary ingredients (specifically chosen among those not yet present in the list) that most frequently co-occur in the platform's recipes.

SmartFridgeService.java

This service implements the Smart Fridge feature, acting as both a dynamic pantry inventory manager and an advanced recommendation engine. Similar to the shopping list, it provides straightforward functionalities to add and remove available ingredients, ensuring strict data consistency by automatically normalizing all inputs to lowercase and validating them against the system's allowed ingredients dictionary.

The core value of this service lies in its integration with the Neo4j graph database: starting from the user's available ingredients, it executes a targeted graph traversal to retrieve recipe suggestions that can be prepared using at least three of the listed items. These results are then sorted in descending order based on the number of matching ingredients. To prevent the severe computational overhead of repeated graph queries, these recommendations are cached in Redis.

To maximize system performance, the service employs a highly tailored cache management strategy. Adding a new ingredient completely invalidates the current suggestions cache, naturally prompting a fresh graph query on the next read; conversely, removing an ingredient triggers a smart, in-place update of the existing cache, actively avoiding full invalidations. Furthermore, if a recommended recipe is deleted from the platform, it is removed from the foodie's cache only upon a view attempt, seamlessly restoring consistency.

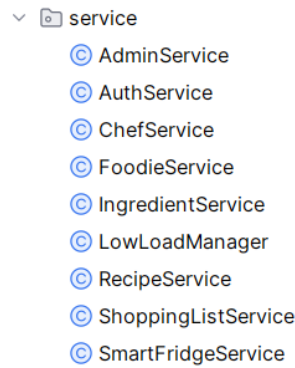


Figure 7.4: Service packages

7.8 Controller

The controller layer is responsible for managing incoming HTTP requests from clients. It functions as the primary entry point of the application, serving as a bridge between client-side requests and the underlying business logic in the service layer. Additionally, the application's endpoints are secured using JSON Web Tokens (JWT) and role-based access control to ensure proper usage and system security.

The following subsections outline the various available controllers and their structural organization.

AdminController.java

Controller to handle administrative endpoints. It facilitates HTTP POST and DELETE requests to approve or reject pending chef registrations and submitted recipes, directly interacting with the AdminService. Additionally, it provides GET endpoints to retrieve paginated lists of pending entities, fetch the full details of submissions before making an approval decision, and retrieve aggregated system analytics, such as user growth metrics, category trend data, and Bayesian chef evaluations.

AuthController.java

Controller to handle authentication endpoints. It exposes POST endpoints for the registration of both chefs and foodies, incorporating preliminary age-validation logic before delegating the process to the AuthService. Furthermore, it provides a unified login endpoint that accepts user credentials and returns a secure JSON Web Token (JWT) required for authorized access to protected application routes.

ChefController.java

Controller to handle chef-specific endpoints. It processes HTTP requests allowing chefs to update strictly permitted personal details (email, password, and birthdate) and to submit new recipes. It acts as the routing layer to the ChefService for managing

the chef's portfolio, exposing endpoints to retrieve paginated views of published, popular, or pending recipes, as well as to access the full details of any specific recipe. Finally, it handles DELETE requests to withdraw pending submissions, remove published recipes, or permanently delete the chef's profile.

FoodieController.java

Controller to handle foodie-specific endpoints. It manages operations related to user profile updates and account deletion. It handles HTTP requests to manage the favourites recipes collections, allowing foodies to add, remove, and retrieve paginated lists of saved recipes, as well as inspecting their full details. Additionally, it exposes endpoints to search for specific chefs by surname and to trigger graph-powered recommendations, retrieving similar recipes or similar culinary creators based on shared ingredients.

IngredientController.java

Controller responsible for managing ingredient-related endpoints. It exposes a GET endpoint to retrieve the complete and exhaustive list of all officially allowed ingredients within the system. This functionality directly supports the client-side application, providing the necessary dataset to guide user inputs via auto-completion or dropdown menus during recipe creation, shopping list management, and smart fridge population.

RecipeController.java

Controller to handle public recipe endpoints. It acts as the primary access point for exploring the platform's approved culinary content. It exposes GET endpoints to fetch the full details of specific recipes and facilitates paginated searches by category, specific chef, or textual title substrings. Additionally, it routes requests to the RecipeService to dynamically retrieve the most popular recipe within each category, optimizing the content discovery experience for all users.

ShoppingListController.java

Controller to handle shopping list operations. It exposes HTTP GET, POST, and DELETE methods that allow foodies to retrieve their current list, add new ingredients, or remove specific items. Furthermore, it provides a dedicated endpoint to retrieve on-demand recommendations, suggesting three complementary ingredients that frequently co-occur in the platform's recipes with the items already present in the user's shopping list.

SmartFridgeController.java

Controller to handle Smart Fridge endpoints. It provides the interface for users to manage their digital pantry, processing requests to add, retrieve, or remove specific ingredients. Most notably, it exposes a specialized GET endpoint that

triggers the SmartFridgeService to compute, cache, and return personalized recipe recommendations based on the current contents of the foodie’s available ingredients.

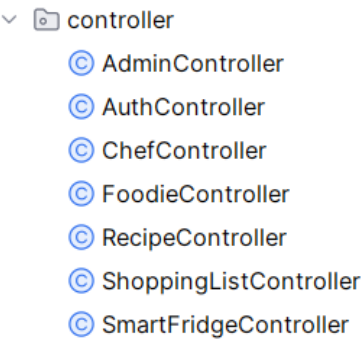


Figure 7.5: Controller packages

7.9 Endpoints

Recipes Endpoints

Recipes Endpoints for viewing and searching recipes (Public Access)			^
GET	/api/recipes/{id}	Get recipe's details	🔒 ▼
GET	/api/recipes/search/{title}/{pageNumber}	Search recipes by title	🔒 ▼
GET	/api/recipes/homeRecipe/{pageNumber}	View newest recipes	🔒 ▼
GET	/api/recipes/chef/{chefId}/{pageNumber}	View chef's recipes	🔒 ▼
GET	/api/recipes/categoryTrend	Retrieve top recipe per category	🔒 ▼
GET	/api/recipes/category/{category}/{pageNumber}	Search by category	🔒 ▼

Ingredients Endpoints

Ingredients Endpoints for managing ingredients allowed in the service			^
GET	/api/ingredients/allowedIngredients	Allowed ingredients	🔒 ▼

Auth Endpoints

Authentication Endpoints for user registration and login			^
POST	/api/auth/register/foodie	Register a new foodie	🔒 ▼
POST	/api/auth/register/chef	Register a new chef	🔒 ▼
POST	/api/auth/login	User login	🔒 ▼

Admin Endpoints

Admin Endpoints for platform administration (Requires ADMIN role)			⌵
POST	/api/admin/approveChef/{username}	Approve a chef registration	🔒 ⌵
POST	/api/admin/approve/{id}	Approve a pending recipe	🔒 ⌵
GET	/api/admin/showRecipes/{pageNumber}	Get pending recipes	🔒 ⌵
GET	/api/admin/showChefs/{pageNumber}	Get pending chef requests	🔒 ⌵
GET	/api/admin/monthlyFoodies/{year}	Get monthly user registration analytics	🔒 ⌵
GET	/api/admin/details/recipe/{recipeId}	Get recipe details	🔒 ⌵
GET	/api/admin/details/chef/{username}	Get chef details	🔒 ⌵
GET	/api/admin/chefsRanking	Get bayesian chef ranking	🔒 ⌵
GET	/api/admin/categoryTrends	Get category trend analytics	🔒 ⌵
DELETE	/api/admin/discardChef/{username}	Decline a chef registration	🔒 ⌵
DELETE	/api/admin/discard/{id}	Discard a pending recipe	🔒 ⌵

Chef Endpoints

Chef Endpoints for managing Chef profiles and their recipes			⌵
POST	/api/chefs/changeInfo	Change chef's information	🔒 ⌵
POST	/api/chefs/addNewRecipe	Submit a new recipe	🔒 ⌵
GET	/api/chefs/showWaiting/{pageNumber}	Show chef's pending recipes	🔒 ⌵
GET	/api/chefs/show/{pageNumber}	Show published recipes	🔒 ⌵
GET	/api/chefs/popular/{pageNumber}	Show popular recipes	🔒 ⌵
GET	/api/chefs/info	Retrieve the chef's personal information	🔒 ⌵
GET	/api/chefs/details/{recipeId}	Get recipe details	🔒 ⌵
DELETE	/api/chefs/removeWaiting/{id}	Remove a pending recipe	🔒 ⌵
DELETE	/api/chefs/deleteRecipe/{id}	Delete an already approved recipe	🔒 ⌵
DELETE	/api/chefs/deleteProfile	Delete chef's profile	🔒 ⌵

Foodie Endpoints

Foodie			Endpoints for managing Foodie profiles and their favorite recipes	^
POST	/api/foodies/changeInfo	Change foodie's personal information	🔒	▼
POST	/api/foodies/addFavourite/{recipeId}	Add a recipe to favourites	🔒	▼
GET	/api/foodies/similarRecipes/{recipeId}	Show similar recipes	🔒	▼
GET	/api/foodies/recipe/{id}	Retrieve saved recipe details	🔒	▼
GET	/api/foodies/matchingChef	Search for chefs by surname	🔒	▼
GET	/api/foodies/info	Retrieve foodie's information	🔒	▼
GET	/api/foodies/getRecipe/{filter}/{pageNumber}	Retrieve favourite recipes filtered	🔒	▼
GET	/api/foodies/findSimilarChef/{chefId}	Show similar chefs	🔒	▼
DELETE	/api/foodies/removeFavourite/{recipeId}	Remove a recipe from favourites	🔒	▼
DELETE	/api/foodies/deleteProfile	Delete foodie's profile	🔒	▼

Shopping List Endpoints

Shopping List			Endpoints for managing the user's shopping list	^
POST	/api/shopping/add	Add ingredients	🔒	▼
GET	/api/shopping/suggestedIngredients	Suggest ingredients	🔒	▼
GET	/api/shopping/get	Retrieve shopping list	🔒	▼
DELETE	/api/shopping/remove	Remove ingredient	🔒	▼

Smart Fridge Endpoints

Smart Fridge			Endpoints for managing the smart fridge and getting recipe recommendations	^
POST	/api/fridge/add	Add ingredients	🔒	▼
GET	/api/fridge/recommendations/{pageNumber}	Get recipe recommendations	🔒	▼
GET	/api/fridge/recipe/{id}	Get recipe details	🔒	▼
GET	/api/fridge/get	Retrieve smart fridge	🔒	▼
DELETE	/api/fridge/remove	Remove ingredient	🔒	▼

7.10 Utils

This subsection presents the utility functions implemented to enhance code maintainability, organization, and reusability. The structural organization of these utility modules is illustrated in Figure 7.6. The subsequent paragraphs provide a detailed description of their respective functionalities.

7.10.1 conversionFunctions

These utility Java classes are dedicated to the transformation and mapping between internal database entities and public-facing Data Transfer Objects (DTOs).

- **ChefUtilityFunctions:** Provides utility methods to map and convert Chef entities and PendingChef registration requests into their corresponding DTOs.
- **FoodieUtilityFunctions:** Handles the conversion between Foodie entities and their respective DTOs.
- **RecipeUtilityFunctions:** Offers methods to convert RecipeMongo entities into specific DTOs. These DTOs are tailored to display different recipe details depending on the user type (chefs or foodies), and to prepare the necessary recipe data before updating the Neo4j graph database.

7.10.2 Parameters

This subsection contains the configuration structures for the application's constant values.

- **Parameters:** A centralized utility class that encapsulates globally accessible static lists of constants. It defines the core category boundaries (e.g., allowed recipe categories or difficulties) and operational constraints of the system, ensuring consistency across both business logic and database queries.

7.10.3 Database Populators

This directory contains initialization scripts designed to populate the application's underlying databases (MongoDB, Neo4j, and Redis) with structured sample data. These methods are not executed by default; they run at application start only if their specific reference flags are set to true within the `application.properties` file, allowing developers to easily seed the environment.

- **FavouritesPopulator:** Randomly generates and assigns favorite recipes to foodie profiles in MongoDB, simultaneously updating the `num_saves` counters across both the `Recipes` and `Chefs` collections. The assignment process is entirely randomized: for each foodie, it generates a random integer between 0 and 200, and subsequently selects that exact number of random recipe IDs to populate the user's saved collection.

- **ChefPopulator:** Interacts with MongoDB to compute and categorize each chef's recipes into "new", "old", and "popular" arrays based on save counts and chronological data, updating the global chef profiles accordingly.
- **IngredientsPopulator:** Reads the `ingredients.txt` file to load all the allowed culinary ingredients into Redis, creating an in-memory list used to quickly validate user inputs.
- **Neo4jPopulator:** Clears the existing graph database and rebuilds it from scratch. It imports chefs, ingredients, and recipes from MongoDB and Redis, and builds the specific nodes and relationships required to power all the platform's recommendation features, including the smart fridge, similar recipe suggestions, similar chef discovery, and shopping list ingredient recommendations.
- **UserListPopulator:** This component handles the initial seeding of the Redis database using data from a local JSON file (`user_lists.json`) to simulate user activity for testing and demonstration purposes. Specifically, it populates Redis with proper users' ShoppingLists and SmartFridge ingredients and expire time. Furthermore, if a user has at least three ingredients in their fridge, the component queries the Neo4j graph database to generate personalized recipe recommendations and caches the results back into Redis.

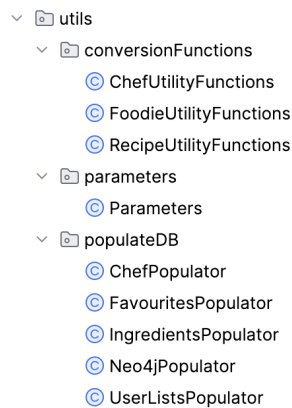


Figure 7.6: Utils packages

Chapter 8

Relevant queries

In this chapter, only the most relevant queries, particularly those involving complex business logic or significant computational load, will be discussed. For this reason, classic CRUD (Create, Read, Update, Delete) operations are not analyzed in detail.

8.1 MongoDB Queries

8.1.1 Chef Ranking using Bayesian Score

This query computes a ranking of chefs based on the popularity of the recipes they have published on the platform. The ranking takes into account both the number of recipes created by each chef and the number of saves received by those recipes. Instead of relying on a simple average, the query applies a Bayesian ranking formula. This approach reduces bias caused by chefs who have very few recipes but unusually high average scores. As a result, the ranking becomes more reliable and statistically meaningful.

Each of the five entries in the result contains the following information:

- **rank:** the position of the chef in the ranking
- **name:** the name of the chef
- **surname:** the surname of the chef
- **score:** the Bayesian popularity score

Aggregation Pipeline Description

The aggregation pipeline performs the following steps:

1. **Match:** The first stage filters the dataset to exclude the *admin* account and the pending chefs, ensuring that only actual chefs are included in the ranking.
2. **AddFields – Handling Missing Values:** In order to avoid null values interfering with calculations, the query replaces missing values with 0 using the `$ifNull` operator for *tot_saves* and *tot_recipes*.
3. **SetWindowFields – Global Statistics Computation:** Computes global statistics (*totalSavesGlobal* and *totalRecipesGlobal*) across the entire dataset to act as a baseline for the formula.

4. **AddFields – Global Average Score:** Calculates the global average score C defined as the ratio between global saves and global recipes. A conditional operator is used to safely set C to 0 if the total global recipes are 0, preventing division-by-zero errors.
5. **AddFields – Individual Average Score:** Computes the individual average score R for each chef (saves per recipe). Similarly, it handles cases where a chef has 0 recipes by defaulting R to 0.
6. **AddFields – Bayesian Score Calculation:** The final score is computed using a Bayesian weighted average formula to prevent statistical noise from dominating the ranking. The aggregation dynamically evaluates the following equation:

$$Score = \left(\frac{N}{N + m} \cdot R \right) + \left(\frac{m}{N + m} \cdot C \right)$$

Where:

- N is the individual chef's total number of recipes (*tot_recipes*).
 - R is the individual chef's average score (saves per recipe).
 - C is the global average score across the entire platform.
 - $m = 5$ is the confidence constant (or prior weight), acting as a stabilizing threshold.
7. **Sort:** The results are sorted in descending order of the computed Bayesian score.
 8. **SetWindowFields – Ranking Generation:** The `$rank` window function automatically assigns a ranking position to each chef based on the score.
 9. **Limit:** The pipeline restricts the output to the top 5 chefs, returning only the highest-ranking profiles.
 10. **Project:** Formats the final output selecting *rank*, *name*, *surname*, and *score*.

```

1 @Aggregation(pipeline = {
2   "{ $match: { username: { $ne: 'admin' }, status: { $ne: 'pending' } } }",
3   "{ $addFields: { " +
4     "   tot_saves: { $ifNull: ['$tot_saves', 0] }, " +
5     "   tot_recipes: { $ifNull: ['$tot_recipes', 0] } " +
6   " } }",
7   "{ $setWindowFields: { partitionBy: null, output: { " +
8     "   totalSavesGlobal: { $sum: '$tot_saves' }, " +
9     "   totalRecipesGlobal: { $sum: '$tot_recipes' } } } }",
10  "{ $addFields: { C: { $cond: [ { $eq: ['$totalRecipesGlobal', 0] }, 0, "
11    +
12    "   { $divide: ['$totalSavesGlobal', '$totalRecipesGlobal'] } ] } } }",
13  "{ $addFields: { R: { $cond: [ { $eq: ['$tot_recipes', 0] }, 0, " +

```

```

13     "    { $divide: ['$tot_saves', '$tot_recipes'] } ] } } }",
14     "{ $addFields: { score: { $add: [ " +
15     "    { $multiply: [ { $divide: ['$tot_recipes', { $add: ['$tot_recipes',
16     5] }] }, '$R' ] }, " +
17     "    { $multiply: [ { $divide: [5, { $add: ['$tot_recipes', 5] }] }, '$C'
18     ] } " +
19     "    ] } } }",
20     "{ $sort: { score: -1 } }",
21     "{ $setWindowFields: { sortBy: { score: -1 }, output: { rank: { $rank:
22     {} } } } }",
23     "{ $limit: 5 }",
24     "{ $project: { rank: 1, name: 1, surname: 1, score: 1 } }"
25 })
26 List<ChefRankAnalyticsDTO> chefBayesianRanking();

```

8.1.2 Category Trend Analysis

This query identifies trends in recipe categories by comparing the publication volume across two distinct time periods to evaluate shifts in popularity. To execute this query, two input dates are provided, dynamically calculated by subtracting exactly one and two years from the current date. Consequently, the analysis compares the following consecutive intervals:

- **Recent Window:** [1 year ago, today]
- **Previous Window:** [2 years ago, 1 year ago]

Each entry in the result represents a recipe category and includes the following information:

- **category:** the recipe category
- **recentCount:** number of recipes created in the recent period
- **previousCount:** number of recipes created in the previous period
- **growthRate:** relative variation between the two periods
- **trendType:** describes the trend of the respective category. It can take one of the following values: NEW, EMERGING, DECLINING, or STABLE.

Aggregation Pipeline Description

The aggregation pipeline performs the following steps:

1. **Match:** The first stage filters the dataset, ensuring that only approved recipes (excluding the *pending* ones) are processed in the subsequent trend calculations.

2. **Time Window Tagging:** Evaluates the *creation_date* of each recipe to determine if it falls within the recent analysis period (*is_recent*) or the baseline comparison period (*is_previous*). According to this logic, recipes created prior to the two-year threshold will have both flags evaluated to **false**, effectively excluding them from the trend counts.
3. **Category Aggregation:** Groups the dataset by recipe category and aggregates the boolean flags to calculate the exact number of recipes published in the recent window versus the previous one.
4. **Trend Calculation (Growth Rate):** Computes the relative variation between the two periods using the standard percentage change formula:

$$\text{Growth Rate} = \frac{\text{recentCount} - \text{previousCount}}{\text{previousCount}}$$

To prevent runtime errors, the pipeline gracefully handles division-by-zero scenarios by returning **null** if the previous period had zero publications. The resulting documents directly map to the *TrendAnalyticsDTO* ready for frontend visualization.

```

1 @Aggregation(pipeline = {
2     "{ $match: { status: { $ne: 'pending' } } }",
3     "{ $addFields: { " +
4         "   is_recent: { $gte: [ { $toDate: '$creation_date' }, ?0 ] }, " +
5         "   is_previous: { $and: [ " +
6             "     { $lt: [ { $toDate: '$creation_date' }, ?0 ] }, " +
7             "     { $gte: [ { $toDate: '$creation_date' }, ?1 ] } " +
8             "   ] } " +
9             "   } }",
10    "{ $group: { " +
11        "   _id: '$category', " +
12        "   recentCount: { $sum: { $cond: [ '$is_recent', 1, 0 ] } }, " +
13        "   previousCount: { $sum: { $cond: [ '$is_previous', 1, 0 ] } } " +
14        " } }",
15    "{ $addFields: { " +
16        "   growthRate: { $cond: [ " +
17            "     { $gt: [ '$previousCount', 0 ] }, " +
18            "     { $divide: [ { $subtract: [ '$recentCount', '$previousCount' ] }, '$previousCount' ] }, " +
19            "     null " +
20            "   ] } " +
21            "   } }"
22  })
23 List<TrendAnalyticsDTO> findCategoryTrend(LocalDateTime recentDate,
    LocalDateTime previousDate);

```

8.1.3 Monthly Foodie Registration Statistics

This query aggregates user registration data on a monthly basis, providing a clear overview of platform adoption rates. By tracking the influx of new Foodies over time, the system enables the Admin to identify periods of peak growth, evaluate seasonal fluctuations, and monitor the overall expansion of the user base.

The resulting document contains the following information:

- **year:** the reference year
- **totalRegisteredFoodies:** total number of users registered in that year
- **monthAnalyticsDTOList:** list of monthly statistics containing the reference month identifiers and the corresponding number of new foodies registered to the application

Aggregation Pipeline Description

The aggregation pipeline performs the following steps:

1. **Match:** The pipeline first filters the dataset based on an input parameter, retaining only the user registrations that occurred during the explicitly requested year.
2. **Monthly Calculation:** The dataset is grouped by month (*YYYY-MM*) to calculate the exact number of new registrations for each specific period.
3. **Intermediate Sorting:** The monthly records are sorted by registration volume in descending order, ranking the months from highest to lowest peaks.
4. **Yearly Grouping and Summation:** The monthly data is subsequently regrouped into macro-categories by year. During this phase, the system calculates the annual sum of registrations (*totalRegisteredFoodies*) and nests the monthly records into a dedicated list. Thanks to the previous sorting stage, the elements inside this list are automatically ordered by registration volume.
5. **Project:** The documents are reshaped to match the expected data transfer object. The internal `_id` field is excluded, and the reference year, originally grouped as a string, is explicitly cast to an integer type using the `$toInt` operator.

```
1 @Aggregation(pipeline = {  
2   "{ $match: { $expr: { $eq: [{ $year: '$registration_date' }, ?0] } } }",  
3   "{ $group: { " +  
4     "_id: { $dateToString: { format: '%Y-%m', date:  
       '$registration_date' } } }, " +  
5     "    year: { $first: { $dateToString: { format: '%Y', date:  
       '$registration_date' } } } }, " +
```

```

6      "          number: { $sum: 1 } " +
7      "    } }",
8      "{ $sort: { 'year': -1, 'number': -1 } }",
9      "{ $group: { " +
10     "        _id: '$year', " +
11     "        totalRegisteredFoodies: { $sum: '$number' }, " +
12     "        monthAnalyticsDTOList: { $push: { month: '$_id', totalFoodies:
13     '$number' } } " +
14     "    } }",
15     "{ $project: { " +
16     "        _id: 0, " +
17     "        year: { $toInt: '$_id' }, " +
18     "        totalRegisteredFoodies: 1, " +
19     "        monthAnalyticsDTOList: 1 " +
20     "    } }"
21 })
YearAnalyticsDTO getMonthlyFoodiesStats(int year);

```

8.1.4 Most Saved Recipe per Category

This query retrieves the most popular approved recipe for each available category on the platform, based on the number of saves received by the foodies. Each entry in the result contains:

- **id**: the unique recipe identifier
- **title**: the recipe title
- **category**: the recipe category
- **imageURL**: the recipe image URL
- **chef**: the embedded object containing the chef's details (e.g., identifier and full name)
- **num_saves**: the total number of saves of the recipe

Aggregation Pipeline Description

The aggregation pipeline performs the following steps:

1. **Match**: Filters the dataset to include only recipes with an *APPROVED* status.
2. **Sort**: Sorts the filtered recipes by their save counter (*num_saves*) in descending order.
3. **Group**: Groups the sorted recipes by category. By utilizing the **\$first** operator, it automatically selects the first recipe in each group, which corresponds to the most saved one due to the previous sorting stage.

4. **Project:** Maps and renames the grouped fields to match the expected format of the final entity.

```
1 @Aggregation(pipeline = {
2     "{ $match: { status: 'APPROVED' } }",
3     "{ $sort: { num_saves: -1 } }",
4     "{ $group: { " +
5         " _id: '$category', " +
6         " recipeId: { $first: '$_id' }, " +
7         " title: { $first: '$title' }, " +
8         " chef: { $first: '$chef' }, " +
9         " num_saves: { $first: '$num_saves' }, " +
10        " image_url: { $first: '$image_url' }" +
11    " } }",
12    "{ $project: { " +
13        " _id: '$recipeId', " +
14        " category: '$_id', " +
15        " title: 1, " +
16        " chef: 1, " +
17        " num_saves: 1, " +
18        " image_url: 1" +
19    " } }"
20 })
21 List<RecipeMongo> findMostSavedRecipePerCategory();
```

8.2 Neo4j Queries

8.2.1 Smart Fridge Recipe Suggestion

This query retrieves recipe recommendations based on a dynamic set of ingredients currently available in the user's smart fridge. The primary objective is to enhance the user experience and reduce food waste by suggesting highly compatible meals.

Indeed, the system takes advantage of the graph structure: it starts from the available ingredients and follows the direct links to all connected recipes. The system then filters these results, keeping only the recipes that are linked to at least three ingredients from the fridge, and ranks them by their compatibility score.

Each entry in the result contains the following information:

- **id:** the unique recipe identifier
- **title:** the recipe title
- **imageURL:** the recipe image URL
- **chefName:** the first name of the authoring chef

- **chefSurname**: the surname of the authoring chef
- **chefId**: the unique identifier of the chef
- **matchCount**: the total number of matching ingredients
- **matchedIngredients**: the explicit list of the matched ingredients

Query Description

The Cypher query performs the following steps:

1. **Graph Traversal (USED_IN)**: The query starts by matching the user's input list with the target **Ingredient** nodes. From there, it follows the outgoing **USED_IN** relationships directly to the connected **Recipe** nodes. It counts how many ingredients are linked to each recipe and collects their names.
2. **Smart Fridge Threshold Filtering**: To make sure the user can actually cook the suggested meal, a filter ensures that the **Recipe** node has at least three active connections ($matchCount \geq 3$) from the ingredients in the fridge.
3. **Author Retrieval (WRITTEN_BY)**: For each validated recipe, the query extends its path by traversing the **WRITTEN_BY** relationship to reach the specific **Chef** node who authored that recipe.
4. **Data Projection**: The query extracts the specific properties from the isolated nodes (**Recipe** details, **Chef** credentials, and matching statistics) to build the final output structure.
5. **Sorting**: Finally, the results are sorted in descending order based on the *matchCount*. This ensures that recipes with the highest number of available ingredients are displayed first.

```

1 @Query("MATCH (i:Ingredient)-[:USED_IN]->(r:Recipe) " +
2     "WHERE i.name IN $myIngredients " +
3     "WITH r, count(i) AS matchCount, collect(i.name) AS
4     matchedIngredients " +
5     "WHERE matchCount >= 3 " +
6     "MATCH (r)-[:WRITTEN_BY]->(c:Chef) " +
7     "RETURN r.mongo_id AS id, " +
8     "      r.title AS title, " +
9     "      r.imageUrl AS imageUrl, " +
10    "      c.name AS chefName, " +
11    "      c.surname AS chefSurname, " +
12    "      c.mongo_id AS chefId, " +
13    "      matchCount, " +
14    "      matchedIngredients " +
15    "ORDER BY matchCount DESC")
16 List<RecipeSuggestionDTO> findRecipesByIngredients(List<String>
    myIngredients);

```

8.2.2 Similar Chefs by Shared Ingredients

This query identifies and recommends chefs with similar culinary styles based on the common ingredients utilized across their respective recipes. To ensure the recommendations are highly relevant, the system explores the shared paths between different Chef nodes while enforcing a strict domain boundary: the shared ingredients must be utilized in recipes belonging to the exact same categories as those of the target chef. Each entry in the result contains the following information:

- **id**: the unique chef's identifier
- **name**: the chef's name
- **surname**: the chef's surname

Query Description

The Cypher query performs the following steps:

1. **Initial Path Expansion and Super-Node Pruning**: The traversal starts from a specific Chef node, following the outgoing `WROTE` and `USES` relationships to map out all the ingredients they use. At this stage, the query applies a graph-specific optimization: it filters out "super-nodes" (generic ingredients linked to 350 or more recipes, like salt or water). This ensures the recommendation is based on unique, meaningful culinary choices rather than generic elements.
2. **Domain Category Aggregation**: Along the traversed path, the query collects the distinct categories of the recipes where the target chef uses that specific ingredient. This ensures contextual accuracy (e.g., if the target chef uses a specific ingredient for desserts, the recommended chef must use it in desserts as well).
3. **Reverse Graph Traversal**: From the filtered Ingredient nodes, the query performs a traversal along the `USED_IN` and `WRITTEN_BY` relationships to find adjacent Recipe and Chef nodes. It ensures that these recipes fall into the previously aggregated contextual categories, and explicitly blocks the path from returning to the original target chef.
4. **Structural Affinity Scoring**: The query calculates a *sharedIngredientsScore* by counting the exact number of distinct, valid ingredient nodes that connect the "other" chefs to the target chef.
5. **Sorting and Subgraph Isolation**: Finally, the stream of potential chefs is sorted in descending order based on this connection score. The query isolates the top 3 Chef nodes with the strongest structural relationship and returns their ID, name, and surname.

```

1 @Query("MATCH (target:Chef {mongo_id:
    $targetMongoId})-[:WROTE]->(r1:Recipe)-[:USES]->(i:Ingredient) " +
2     "WITH target, i, collect(DISTINCT r1.category) AS targetCategories " +
3     "WHERE COUNT { (i)-[:USED_IN]->() } < 350 " +
4     "WITH DISTINCT target, i, targetCategories " +
5     "MATCH (i)-[:USED_IN]->(r2:Recipe)-[:WRITTEN_BY]->(other:Chef) " +
6     "WHERE target <> other AND r2.category IN targetCategories " +
7     "WITH other, count(DISTINCT i) AS sharedIngredientsScore " +
8     "ORDER BY sharedIngredientsScore DESC " +
9     "LIMIT 3 " +
10    "RETURN other.mongo_id AS id, " +
11    "        other.name AS name, " +
12    "        other.surname AS surname")
13 List<ChefInfoDTO> findSimilarChefs(@Param("targetMongoId") String
    targetMongoId);

```

8.2.3 Similar Recipes

This query finds and suggests similar recipes based on the highest number of shared ingredients, facilitating content discovery for users browsing a specific dish. The system analyzes how many **Ingredient** nodes act as a direct bridge between the recipe the user is currently browsing and other dishes in the database. From a domain perspective, this enhances content discovery, allowing food enthusiasts to explore new variations of a dish with a highly overlapping flavor profile.

Each entry in the result contains the following information:

- **id**: the recipe's unique identifier
- **title**: the recipe's title
- **imageUrl**: the recipe's image URL
- **chefName**: the chef's name
- **chefSurname**: the chef's surname
- **chefId**: the chef's unique identifier
- **matchCount**: the number of matched ingredients
- **matchedIngredients**: the list of matched ingredients

Query Description

The Cypher query performs the following steps:

1. **Match Ingredients:** The query initiates traversal starting from the target **Recipe** node. It first follows the outgoing **USES** relationships to reach its specific **Ingredient** nodes, and then immediately traverses backwards along the incoming **USED_IN** relationships to find all adjacent recipes that share those exact same ingredients.
2. **Filter:** To prevent a cyclic path back to the starting point, the query explicitly excludes the original target **Recipe** node from the results, avoiding self-referencing.
3. **Aggregation:** Counts the total number of shared ingredients (*sharedIngredientsCount*) and collects their names into a list.
4. **Match Chef:** From the newly identified recipe nodes, the query extends its path by traversing the outgoing **WRITTEN_BY** relationship to fetch the authoritative **Chef** node who created each dish.
5. **Sort and Limit:** Orders the results in descending order by the match count, using the recipe title as a secondary ascending sort, and limits the output to the top 3 similar recipes.

```

1 @Query("MATCH (target:Recipe {mongo_id:
    $recipeId})-[:USES]->(i:Ingredient)-[:USED_IN]->(other:Recipe) " +
2     "WHERE target <> other " +
3     "WITH other, count(i) AS sharedIngredientsCount, collect(i.name) AS
    sharedIngredients " +
4     "MATCH (other)-[:WRITTEN_BY]->(c:Chef) " +
5     "RETURN other.mongo_id AS id, " +
6     "    other.title AS title, " +
7     "    other.imageURL AS imageURL, " +
8     "    c.name AS chefName, " +
9     "    c.surname AS chefSurname, " +
10    "    c.mongo_id AS chefId, " +
11    "    sharedIngredientsCount AS matchCount, " +
12    "    sharedIngredients AS matchedIngredients " +
13    "ORDER BY matchCount DESC, other.title ASC " +
14    "LIMIT 3")
15 List<RecipeSuggestionDTO> findSimilarRecipes(String recipeId);

```

8.2.4 Suggested Complementary Ingredients

Given a list of ingredients currently in a user's shopping list, this query suggests complementary items by analyzing recipe data. The system treats recipes as connecting hubs: it starts from the selected **Ingredient** nodes and traverses the graph to find neighboring ingredients that frequently share the same recipe context. From a domain perspective, this uncovers natural culinary affinities and flavor pairings, helping users optimize their grocery shopping by suggesting items they might have

forgotten or that pair perfectly with what they are buying, while simultaneously filtering out common ignored ingredients. Each entry in the result contains the following information:

- **originalIngredient**: the name of the original ingredient from the shopping list.
- **suggestedIngredients**: a list of the top 3 ingredients that appear most frequently in recipes alongside the original one.

Query Description

The Cypher query performs the following steps:

1. **Match Co-occurrences**: The query starts from the target **Ingredient** nodes matching the shopping list: it follows the outgoing **USED_IN** relationships to find all host **Recipe** nodes; from there, it immediately branches out through the outgoing **USES** relationships to discover all other ingredient nodes present in those same dishes.
2. **Self-Suggestion Filtering**: To ensure the suggestions are meaningful and not redundant, the query filters out a dynamic list of ignored ingredients (*\$ignoredIngredients*). At the application level, the backend constructs this list by combining common ingredients like salt or water, with the items already present in the user's shopping list. This completely prevents self-referencing and redundant suggestions.
3. **Path Strength Counting**: The query groups the data by the original ingredient and the discovered neighbor, counting the exact number of recipe nodes (*occurrences*) that act as a bridge between them. This count measures the strength of the culinary association.
4. **Graph Slicing and Projection**: Finally, the pairs are sorted in descending order based on their connection frequency. The query uses graph collection slicing (`[0..3]`) to isolate only the top 3 most relevant complementary ingredients for each item on the list.

```
1 @Query("MATCH
2   (target:Ingredient)-[:USED_IN]->(r:Recipe)-[:USES]->(other:Ingredient) " +
3   "WHERE target.name IN $ingredientList " +
4   "AND NOT other.name IN $ignoredIngredients " +
5   "WITH target.name AS originalIngredient, other.name AS
6   suggestedIngredient, count(r) AS occurrences " +
7   "ORDER BY occurrences DESC " +
8   "RETURN originalIngredient, collect(suggestedIngredient)[0..3] AS
9   suggestedIngredients")
List<IngredientSuggestionDTO> findSuggestedIngredientsForList(
    @Param("ingredientList") List<String> ingredientList,
    @Param("ignoredIngredients") List<String> ignoredIngredients);
```

8.3 Redis Queries and Caching Strategy

8.3.1 Key Patterns

To ensure efficient data retrieval and maintain logical separation within Redis data, a strict key naming convention has been adopted. As described in Section 6.3, the keys are structured as follows:

```
1 //Smart Fridge and Shopping List key patterns
2 REDIS_KEY_PREFIX = "Foodie:";
3
4 REDIS_KEY_SUFFIX = ":shoppingList";
5 REDIS_KEY_FRIDGE_SUFFIX = ":smartFridge:ingredients";
6 REDIS_KEY_RECIPES_SUFFIX = ":smartFridge:suggestions";
7
8 //Allowed ingredients key pattern
9 INGREDIENTS_REDIS_KEY = "Allowed_ingredients";
```

8.3.2 Ingredients Management

To enforce strict data consistency and validation across the platform, a specialized global Redis key named `Allowed_ingredients` was implemented. Structured as a Redis *Set*, this caching layer serves as a centralized dictionary containing the complete list of authorized and recognized ingredients allowed within the application. Programmatic interactions with this data structure are encapsulated and managed entirely via the `IngredientService` class.

This centralized validation mechanism plays a critical, multi-purpose role across the system. It ensures that any ingredient added by users to their *Smart Fridge* or *Shopping List*, or by the chef during the creation of his recipe, maps perfectly to a standardized entry, preventing arbitrary text or typos.

In particular, the validation operations are managed using the following method that queries the `Allowed_ingredients` set directly with the Redis `SISMEMBER` command.

```
1 public boolean isValidIngredient(String ingredientName) {
2     // ...
3     try (Jedis jedis = jedisSentinelPool.getResource()) {
4         return jedis.sismember(INGREDIENTS_REDIS_KEY,
5                               ingredientName.toLowerCase().trim());
6     }
7     // ...
8 }
```

Furthermore, the application needs to retrieve the complete set of allowed ingredients to populate the frontend autocomplete options. This is achieved by querying the `Allowed_ingredients` set using the `SMEMBERS` command:

```

1 public Set<String> getAllAllowedIngredients() {
2     // ...
3     try (Jedis jedis = jedisSentinelPool.getResource()) {
4         return jedis.smembers(INGREDIENTS_REDIS_KEY);
5     }
6     // ...
7 }

```

8.3.3 Shopping List Management

Each user's Shopping list is stored as a Redis Set. This data structure prevents duplicate entries while providing efficient insertion, removal, and retrieval operations.

Retrieve Shopping List

To retrieve all ingredients currently stored in a user's shopping list, the application executes the `SMEMBERS` command:

```

1 private IngredientsListDTO returnShoppingList(String username) {
2
3     String key = REDIS_KEY_PREFIX + username + REDIS_KEY_SUFFIX;
4
5     // ...
6     try (Jedis jedis = jedisSentinelPool.getResource()) {
7         ingredients = jedis.smembers(key);
8     }
9     // ...
10    return ingredientsListDTO;
11 }

```

The following example illustrates the output obtained by querying the database directly through the Redis Command Line Interface (Redis CLI), referencing the profile of the user with the username `sierra.mitchell`:

```

1 SMEMBERS "Foodie:sierra.mitchell:shoppingList"
2 1) "garlic"
3 2) "kombu"
4 3) "parmesan cheese"
5 4) "tomatoes"
6 5) "basil"

```

Add Ingredients

One or more valid ingredients can be added at the same time using the `SADD` command. Before insertion, all ingredient names are normalized to lowercase to ensure consistency. After the update, a TTL of 15 days (1296000 seconds) is assigned through the `EXPIRE` command.


```

1 public IngredientsListDTO addIngredients(List<String> ingredients) {
2
3     // ...
4     ingredients.removeIf(Objects::isNull);
5     ingredients.removeIf(ingredient ->
6         !ingredientService.isValidIngredient(ingredient));
7     ingredients.replaceAll(String::toLowerCase);
8
9     String key = REDIS_KEY_PREFIX + foodie.getUsername() + REDIS_KEY_SUFFIX;
10
11     if (!ingredients.isEmpty()) {
12         try (Jedis jedis = jedisSentinelPool.getResource()) {
13             jedis.sadd(key, ingredients.toArray(new String[0]));
14             jedis.expire(key, 86400*15);
15         }
16     }
17     // ...
18     return returnShoppingList(foodie.getUsername());
19 }

```

It is worth noting that each time a user adds an ingredient, the Time-To-Live (TTL) counter is automatically reset. This mechanism allows the system to dynamically identify active foodies and preserve their data.

Remove Ingredient

A specific ingredient can be removed from the shopping list using the SREM command. Following the exact same reasoning applied to the ingredient insertion function, the TTL is refreshed during deletion as well to ensure that actively maintained lists are retained in the cache.

```

1 public IngredientsListDTO removeIngredient(String ingredient) {
2
3     // ...
4     if(ingredientService.isValidIngredient(ingredient.toLowerCase())) {
5         String key = REDIS_KEY_PREFIX + foodie.getUsername() +
6             REDIS_KEY_SUFFIX;
7         try (Jedis jedis = jedisSentinelPool.getResource()) {
8             jedis.srem(key, ingredient.toLowerCase());
9             jedis.expire(key, 86400*15);
10         }
11     }
12     return returnShoppingList(foodie.getUsername());
13 }

```

8.3.4 Smart Fridge Management

The ingredients currently available in a user's Smart Fridge are stored as a Redis Set. This choice guarantees uniqueness while allowing efficient simple operations. Any time the list is modified because new ingredients are added or an ingredient is removed, the TTL (initially set to 15 days) is restored.

Retrieve Fridge Ingredients

To retrieve all ingredients currently available in the Smart Fridge, the application executes the SMEMBERS command:

```
1 private IngredientsListDTO returnSmartFridge(String username) {
2
3     String key = REDIS_KEY_PREFIX + username + REDIS_KEY_FRIDGE_SUFFIX;
4     Set<String> ingredients;
5     try (Jedis jedis = jedisSentinelPool.getResource()) {
6         ingredients = jedis.smembers(key);
7     }
8
9     // ...
10    return ingredientsListDTO;
11 }
```

The following example illustrates the output obtained by querying the database directly through the Redis CLI:

```
1 SMEMBERS "Foodie:sierra.mitchell:smartFridge:ingredients"
2 1) "tomatoes"
3 2) "onions"
4 3) "cucumbers"
5 4) "lettuce"
```

Add Ingredients and Invalidate Cache

When new valid ingredients are added, they are stored using SADD and the TTL is refreshed. Since the set of available ingredients directly affects recipe recommendations, the cached suggestions are immediately invalidated through DEL, forcing a new Neo4j query on the next request.

```
1 public IngredientsListDTO addIngredients(List<String> ingredients) {
2
3     // ...
4     String key = REDIS_KEY_PREFIX + foodie.getUsername() +
5     REDIS_KEY_FRIDGE_SUFFIX;
6
7     if (!ingredients.isEmpty()) {
8         try (Jedis jedis = jedisSentinelPool.getResource()) {
9             jedis.sadd(key, ingredients.toArray(new String[0]));
10        }
```

```

9         jedis.del(REDIS_KEY_PREFIX + foodie.getUsername() +
10                 REDIS_KEY_RECIPES_SUFFIX);
11         jedis.expire(key, 86400*15);
12     }
13 }
14 // ...
15 return returnSmartFridge(foodie.getUsername());
16 }

```

Remove Ingredient

Ingredients removed from the fridge are deleted from the set via the SREM command, followed by a reset of the TTL counter. Additionally, this operation triggers an update of the cached recipe suggestions through the invocation of the `updateCacheAfterRemoval()` function, which is detailed in Section 8.3.5.

```

1 public IngredientsListDTO removeIngredient(String ingredient) {
2
3     // ...
4     if(ingredientService.isValidIngredient(ingredient)) {
5         String key = REDIS_KEY_PREFIX + foodie.getUsername() +
6                 REDIS_KEY_FRIDGE_SUFFIX;
7         try (Jedis jedis = jedisSentinelPool.getResource()) {
8             jedis.srem(key, ingredient);
9             jedis.expire(key, 86400*15);
10        }
11        updateCacheAfterRemoval(foodie.getUsername(), ingredient);
12    }
13    return getSmartFridge();
14 }

```

8.3.5 Recipe Suggestions Caching

To reduce the load on the Neo4j graph database and improve response times, recipe recommendations generated by the Smart Fridge feature are cached in Redis as a serialized JSON string.

Retrieve Cached Suggestions

Before executing the graph query that computes recipe recommendations, the application checks whether cached suggestions already exist on Redis by using the GET command.

```

1 public SliceRecipeDTO<RecipeSuggestionDTO> getRecommendations (String
2     username, int pageNum) {
3     // ...
4     String cacheKey = REDIS_KEY_PREFIX + username + REDIS_KEY_RECIPES_SUFFIX;
5     String json;

```

```

5      try (Jedis jedis = jedisSentinelPool.getResource()) {
6          json = jedis.get(cacheKey);
7      }
8
9      if (json != null) {
10         try {
11             suggestions = objectMapper.readValue(json,
12                 new TypeReference<>(){});
13         }
14         // ...
15     }
16     // ...
17     return new SliceRecipeDTO<>(content, hasNext, hasPrevious);
18 }

```

The following examples illustrates the output obtained by querying the database directly through the Redis CLI ¹:

```

1 GET "Foodie:sierra.mitchell:smartFridge:suggestions"
2 [
3     {
4         "id" : "69cea24e14f896db429ba7c8",
5         "title" : "Grilled Gyro Burgers",
6         "imageUrl" : "https://grilled-gyro-burgers.jpg",
7         "chef" : "Martha Collison",
8         "matchCount" : 4,
9         "matchedIngredients" : [ "tomatoes", "onions", "cucumbers", "lettuce" ],
10
11         "chef_id" : "69ce825a14f896db429a149b"
12     },
13     // ...
14 ]

```

Cache New Suggestions

In case of a cache miss, Neo4j is queried and the resulting list of *RecipeSuggestionDTO* objects is serialized into JSON through Jackson's *ObjectMapper*. The serialized data is then stored in Redis using SETEX, which simultaneously assigns a TTL of 15 days to the corresponding key.

```

1 public SliceRecipeDTO<RecipeSuggestionDTO> getRecommendations(String
2     username, int pageNum) {
3
4     // ...
5     String cacheKey = REDIS_KEY_PREFIX + username + REDIS_KEY_RECIPES_SUFFIX;
6     String json;
7     try (Jedis jedis = jedisSentinelPool.getResource()) {

```

¹The result has been obtained with redis-cli -raw flag in order to see the JSON format

```

7         json = jedis.get(cacheKey);
8     }
9
10    List<RecipeSuggestionDTO> suggestions;
11    if (json != null) {
12        // ...
13    }
14    else {
15        // ...
16        suggestions =
17            recipeNeo4jRepository.findRecipesByIngredients(ingredients);
18        // ...
19
20        try (Jedis jedis = jedisSentinelPool.getResource()) {
21            jedis.setex(cacheKey, 86400*15, objectMapper.
22
23        writerWithDefaultPrettyPrinter().writeValueAsString(suggestions));
24        }
25        // ...
26    }
27    return new SliceRecipeDTO<>(content, hasNext, hasPrevious);
28 }

```

Smart Cache Update on Ingredient Removal

To avoid unnecessary database queries, ingredient removal does not automatically invalidate the cache. Instead, the application retrieves the cached JSON using GET, updates the match count of the affected recipes in memory, removes recipes that no longer satisfy the minimum matching threshold (three ingredients), and stores the updated cache using SETEX with a refreshed TTL. If no suggestions remain, the cache entry is deleted.

Due to space constraints, the source code for this functionality has been omitted from this document. To review the implementation, please refer to the `updateCacheAfterRemoval()` function defined in `SmartFridgeService.java`.

Cache Update on Missing Recipe Selection

If a Foodie wants to visualize the details of a recipe suggestion whose corresponding recipe is no longer available in MongoDB, the application retrieves the cached suggestions through GET, removes the unavailable recipe from the list, and stores the updated cache again using SETEX command.

Also in this case, due to space constraints, the source code for this functionality has been omitted from this document. To review the implementation, please refer to the `getFridgeRecipeById()` function defined in `SmartFridgeService.java`.

Chapter 9

Design choices

9.1 Indexes

9.1.1 MongoDB Indexes

This section explores the indexes that we implemented to improve the performance of read operations on the MongoDB collections. For each candidate index, we evaluated the corresponding application query and compared its execution with and without the index. The comparison was performed using MongoDB Compass, considering the execution time, the number of examined index keys and the number of examined documents.

The purpose of this analysis is not only to identify useful indexes, but also to avoid unnecessary indexes that would introduce additional overhead during insert and update operations.

9.1.2 Recipes Collection

In this collection, in addition to the default index on the `_id` field, specific indexes were defined on the following fields to optimize database performance:

- **title (Text Index)**: A text index designed to optimize full-text search operations based on the recipe's title.
- **category (Standard Index)**: A standard index used to speed up queries that filter recipes by their category.
- **creation_date (Descending Index)**: A descending index implemented to drastically optimize the retrieval of the newest recipes.

Category

An index on the `category` field was added to improve the performance of category-based recipe browsing. This operation is frequently used when users filter recipes according to a specific category, such as Desserts, Main courses or Appetizers.

To evaluate the usefulness of this index, we executed a query that retrieves all recipes belonging to the "Desserts" category, both with and without the index.

```
{ category: "Desserts" }
```

	Without index	With index
Execution Time (ms)	96	19
Keys examined	0	3011
Documents examined	15832	3011

Table 9.1: Performance comparison for the **category** index in the Recipes collection

Without the index, MongoDB performed a full collection scan, examining all 15832 documents in the **Recipes** collection to find the matches. By leveraging the **category** index, MongoDB only examined the 3011 documents that strictly satisfied the query condition.

The results demonstrate a significant performance improvement: the execution time dropped from 96 ms to 19 ms, and the number of examined documents was drastically reduced. Consequently, the index on the **category** field proved to be highly beneficial and was permanently integrated into the database schema.

Substring and Multi-Word Case-Insensitive Search

The **title** field is heavily utilized by the application's search engine. Users frequently search for recipes using partial keywords or multi-word phrases rather than exact titles. At the repository layer, this logic is handled by the Spring Data method:

```
findByTitleContainingIgnoreCase(String title)
```

This derived query method translates into a case-insensitive regular expression search in MongoDB. To investigate the performance limitations of this approach when searching for a compound multi-word string, a test query was executed without index on the **title** field using the equivalent regular expression:

```
{ title: { $regex: "Upside-down cake", $options: "i" } }
```

The execution plan yielded the following metrics:

	Substring Search (Without Index)
Execution Time (ms)	58
Index Keys Examined	0
Documents Examined	15832

Table 9.2: Performance metrics for case-insensitive substring search without an index

As shown in Table 9.2, without an index, MongoDB is forced to perform a full collection scan. The query engine had to load and examine all 15832 documents from disk into memory just to return the 28 matching records. While an execution time of 58 ms may appear acceptable for a small dataset, this approach represents a major architectural bottleneck because both execution time and hardware resource consumption scale linearly with the growth of the collection.

Optimizing via MongoDB Text Index To resolve this scaling issue and efficiently handle phrase searches, a dedicated MongoDB Text Index was implemented by adding to the `title` field the Spring Data’s `@TextIndexed` annotation. This index automatically tokenizes the text, stems words to their roots, and filters out common stop-words.

To search for the exact sequence of multiple words while ignoring casing, the query was executed using MongoDB’s native `$text` operator, wrapping the target phrase in escaped double quotes:

```
{ $text: { $search: "\"Upside-down cake\"" } }
```

The tested query with the text index active generated the following results:

	Phrase Search (With Text Index)
Execution Time (ms)	22
Index Keys Examined	1110
Documents Examined	1060

Table 9.3: Performance metrics for phrase match utilizing the MongoDB Text Index

Comparing the metrics highlights a substantial performance optimization. The integration of the text index reduced the query execution time from 58 ms to 22 ms. Instead of scanning all 15832 entries in the database, MongoDB leveraged the tokenized index structure to isolate the specific terms, examining only 1110 index keys and loading a subset of 1060 documents into memory to evaluate the exact phrase matching.

These empirical results lead us to maintain the MongoDB Text Index, since it improves overall performance and ensures that the application’s search engine remains highly scalable as the recipe collection grows.

Creation Date

A descending index on the `creation_date` field was added to drastically optimize the retrieval of the newest recipes. This operation is frequently executed on the application’s homepage, where users are presented with the most recently published content, requiring chronological sorting and pagination (limiting the results to 5 items per page).

To evaluate the usefulness of this index, we executed a query on MongoDB Shell that retrieves the top 5 most recent recipes, both with and without the descending index.

```
db.recipes.find({}).sort({ creation_date: -1 }).limit(5)
```


	Without index	With index
Execution Time (ms)	73	2
Keys examined	0	5
Documents examined	15832	5

Table 9.4: Performance comparison for the `creation_date` descending index

Without the index, MongoDB was forced to perform a full collection scan. It examined all 15832 documents in the `Recipes` collection, loaded them into memory to sort them chronologically, and only then returned the top 5 results. By leveraging the descending index on `creation_date`, MongoDB immediately accessed the first 5 index keys and examined exclusively the 5 corresponding documents, bypassing the in-memory sort entirely.

The results demonstrate a massive performance improvement: the execution time went from 73 ms to a nearly instantaneous 2 ms. Furthermore, the number of examined documents was reduced to the absolute minimum required by the query limit. Consequently, this descending index proved to be essential for the application’s scalability and was permanently integrated into the database schema.

9.1.3 Chef Collection

In this collection, in addition to the default index on the `_id` field, an index was considered on the following fields:

- **Username (Unique index):** A unique index to ensure each username is uniquely associated to a Chef.
- **Surname (Text index):** A text index designed to optimize full-text search operations based on the chef’s surname.

Username

An index on the `username` field was added to improve frequent login, registration and profile retrieval operations.

To evaluate the efficiency of this index, we tested an exact-match query looking for a specific chef profile, comparing execution plans before and after the index integration:

```
{ username: "nigella.lawson" }
```

	Without index	With index
Execution Time (ms)	0	0
Keys examined	0	1
Documents examined	94	1

Table 9.5: Performance comparison for the `username` index in the `Chefs` collection

Although the execution time remained at 0 ms in both scenarios due to the currently small dataset (94 documents), the benefit of the index is clearly visible: the database goes from reading the entire collection to instantly finding the exact document needed. While this efficiency will become essential to keep the system fast as the application grows, we decided to implement and keep this index immediately for a more fundamental reason. Being a *Unique Index*, it automatically prevents the creation of duplicate chef profiles, guaranteeing data consistency directly at the database level.

Surname

Since the application allows *foodies* to search for chefs by their last name (either partial or complete), we evaluated the introduction of a *Text Index* on the `surname` field to support this feature. At the repository layer, the logic is handled by the following Spring Data method:

```
findBySurnameContainingIgnoreCase(String chefSurname)
```

To evaluate the usefulness of this index, we searched for a chef's surname both with and without the index. Since a *Text Index* requires a specific MongoDB syntax, the queries executed in the two scenarios were respectively:

```
// Query without index
{ surname: { $regex: "lawson", $options: "i" } }

// Query with text index
{ $text: { $search: "\"lawson\"" } }
```

	Without index	With index
Execution Time (ms)	1	1
Keys examined	0	1
Documents examined	94	1

Table 9.6: Performance comparison for the `surname` index in the Chefs collection

Without the index, MongoDB performed a collection scan and examined all 94 documents in the `chefs` collection. With the index, MongoDB examined only one index key and one document to retrieve the requested chef.

Although the index is correctly used by MongoDB, the execution time does not show a significant improvement because the current collection is small. For this reason, the index on `surname` was not considered strictly necessary in the current version of the application. However, it could become useful in a future scenario where the number of chefs grows and surname-based search becomes more a frequent operation.

9.1.4 Foodie Collection

The Foodies collection stores data related to the standard users of the platform. Although these users perform a high volume of read operations across the application (e.g., browsing recipes or chefs), the system does not include any functionality that requires searching for Foodie profiles. For these reasons, the indexes defined for this collection are the following:

- **_id (Default index):** The default MongoDB index, automatically generated for document identification.
- **Username (Unique index):** A unique index to ensure each username is uniquely associated to a Chef.

Username

Although Foodies are not searchable entities within the platform, the index on the username field was strictly necessary for two main reasons. First, it facilitates and optimizes the login operations, analogous to the architectural choice described for the Chefs in Section 9.1.3. Second, and most importantly, it acts as a fundamental database-level guardrail for data integrity. Configured as a Unique Index, it guarantees absolute uniqueness during the registration workflow, automatically preventing the insertion of duplicate profiles and ensuring that two Foodies can't register with the exact same username.

9.2 Neo4j Graph Database Indexes

This section explores the indexes that we implemented to improve the performance of read operations on the Neo4j nodes. For each candidate index, we have evaluated the corresponding application query and compared its execution with and without the index. The comparison was performed using Neo4j Desktop with the Cypher keyword "PROFILE", considering the number of nodes and relationships examined and the execution time.

In the next sessions we are going to consider the following indexes:

- **ingredient_name_idx:** an index on the `name` field for the `Ingredient` node.
- **recipe_mongo_id_idx:** an index on the `mongo_id` for the `Recipe` node.
- **chef_mongo_id_idx:** an index on the `mongo_id` for the `Chef` node.

9.2.1 ingredient_name_idx

To optimize read operations requiring graph traversal starting from specific ingredients, an index was introduced on the `name` field of the `Ingredient` nodes (named `ingredient_name`). This architectural design choice prevents the database engine from executing a full linear scan of the entire graph (*AllNodesScan*) to locate the starting nodes.

Smart Fridge Recipe Suggestion

The benefits of this optimization are particularly evident in the *Smart Fridge Recipe Suggestion* query (explained in Section 8.2.1), which returns a list of recipes that can be prepared using at least 3 ingredients from the provided input list, sorting the results in descending order by the number of `matchingIngredients`. The analysis was performed providing the following input parameters:

Available ingredients: ["butter", "bread", "avocados", "spaghetti pasta"]

Execution without index In the absence of an index, the execution plan highlighted the following sequential steps:

- **18,859 nodes scanned:** the system processes the totality of the nodes present in the database to identify the targets.
- **4 Ingredient nodes identified:** isolated only at the end of the complete scan.
- **5,532 relationships examined:** traversal of the outgoing edges (`USED_IN`) starting from the 4 ingredients to reach the recipe nodes.
- **5,253 unique recipes evaluated:** the engine reaches and aggregates the target recipes.
- **4 final recipes returned:** the final results satisfying the logical constraint of "at least 3 matching ingredients".

The total cost of this unoptimized operation, as documented by the profiling tool, amounts to *51,733 DB Hits*, completed in *53 ms*.

Execution with index By executing the same query leveraging the `ingredient_name_idx` index, the initial search phase is radically optimized:

- **Direct access (4 nodes):** instead of iterating over 18,859 nodes, the execution exploits the index data structure to instantly access the required nodes, from which the traversal is then initiated.
- **11,080 DB Hits:** the number of database accesses drops drastically.
- **28 ms execution time:** the overall response time is nearly halved.

As highlighted in Table 9.7, the introduction of the index significantly improves system performance, reducing graph navigation operations by nearly 80%.

Evaluated Metric	Without Index	With Index
Initial search phase	18,859 nodes	4 nodes
Computational cost	51,733 DB Hits	11,080 DB Hits
Execution time	53 ms	28 ms

Table 9.7: Performance comparison of the Smart Fridge Recipe Suggestion query.

9.2.2 recipe_mongo_id_idx

To optimize read operations targeting specific recipes, an index was introduced on the `mongo_id` property of the `Recipe` node. Similar to the previous index, this optimization avoids a full node scan when locating the starting node.

Similar Recipes Suggestion

The benefits of this optimization are particularly evident in the *Similar Recipe Suggestion* query (explained in Section 8.2.3). The analysis evaluated the traversal path required to find the top three matching recipes starting from a single target recipe identifier. For this benchmark, the system was provided in input with the *"Pumpkin Roll"* recipe (`mongo_id = "69cea24b14f896db429b6e03"`).

Execution without index In the absence of an index, the execution plan highlighted the following sequential steps:

- **18,859 nodes scanned:** the system is forced to perform a full database scan to locate the starting recipe, generating nearly 40,000 DB hits just for the initial filtering.
- **1 target Recipe isolated:** the unique starting point is identified only at the end of the complete scan.
- **10 relationships traversed (USES):** the engine expands the target recipe to discover its 10 specific ingredients.
- **15,449 relationships traversed (USED_IN):** following a traversal from the 10 ingredients, the system explores over 15,400 edges to find all other recipes sharing those ingredients.
- **8,228 unique recipes aggregated:** the engine groups the valid paths, discovering over 8,200 unique recipes that share at least one ingredient with the target.
- **3 final recipes and the corresponding chefs:** an optimized *Sort* and *Limit* operation immediately isolates the absolute Top 3 matches, and the final `WRITTEN_BY` traversal is efficiently executed only for the winning recipes.

The total cost of this un-optimized operation amounts to *79,948 DB Hits*, completed in *100 ms*.

Execution with index By executing the same query leveraging the `recipe_mongo_id_idx` index, the initial bottleneck is entirely resolved:

- **Direct access (1 node):** instead of scanning 18,859 nodes, the execution engine exploits the index structure to instantly access the exact target recipe.
- **39,296 DB Hits:** the number of overall database accesses is halved.
- **62 ms execution time:** the execution time is significantly reduced.

As highlighted in Table 9.8, resolving the initial search phase through indexing cuts the overall query workload by 40%. The residual execution time is structurally inherent to the dataset due to the massive volume of connections branching out from common ingredients. However, testing a recipe with 10 ingredients accurately reflects the graph engine’s performance under highly realistic load conditions, as the platform’s overall average stands at approximately 9.94 ingredients per recipe.

Evaluated Metric	Without Index	With Index
Initial search phase	18,859 nodes scanned	Direct access (1 node)
Computational cost	79,948 DB Hits	39,296 DB Hits
Execution time	100 ms	62 ms

Table 9.8: Performance comparison of the Similar Recipes Suggestion query.

9.2.3 chef_mongo_id_idx

To optimize read operations targeting specific chefs, an index was introduced on the `mongo_id` property of the Chef node.

Find Similar Chef

The benefits of this optimization are particularly evident in the *Find Similar Chef* query (explained in Section 8.2.2). Starting from the target chef, we traverse through their ingredients to find other highly correlated chefs. To test the query, the `mongo_id` of chef Gordon Ramsay was used as the target parameter (`mongo_id = "69ce825a14f896db429a14a5"`).

Execution without index In the absence of an index, the execution plan highlighted the following sequential steps:

- **18,859 nodes scanned:** the system is forced to perform a full database scan to locate the starting Chef node, generating a total of 37,812 DB hits just for the initial scan and filtering operations.
- **1 target Chef isolated:** the unique starting point is identified only at the end of the complete scan.

- **Over 56,900 paths generated:** the system performs a massive expansion, representing the structural bottleneck of the query. It traverses through `WROTE`, `USES`, `USED_IN`, and `WRITTEN_BY` edges to find other interconnected chefs, consuming over 546,000 DB hits in the process.
- **Paths filtered and aggregated:** the engine processes the generated paths, identifying distinct ingredients and narrowing them down to 92 unique correlated chefs.
- **3 final rows returned:** an optimized *Top* operation finalizes the execution, ordering by *sharedIngredientsScore* and returning only the top 3 similar chefs.

The total cost of this un-optimized operation amounts to 661,062 DB hits.

Execution with index By executing the same query leveraging the index on the `mongo_id` property of the `Chef` node, the initial bottleneck is entirely resolved:

- **1 exact node accessed:** the global scan and manual filtering are replaced by a targeted *NodeIndexSeek* on the `Chef(mongo_id)` index, instantly locating the target chef with a mere 2 DB hits.
- **Over 56,900 paths generated:** the subsequent graph traversal remains perfectly identical to the non-indexed execution, confirming that the deep expansion is the core workload of the query.
- **Paths filtered and aggregated:** the resulting paths are successfully grouped and ranked to isolate the unique chefs.
- **3 final rows returned:** the optimized plan isolates the top 3 results smoothly.

While the deep traversal remains resource-intensive, the introduction of the index saves 37,810 DB hits right at the beginning of the execution, eliminating the *AllNodesScan* and dramatically optimizing the starting node lookup.

Evaluated Metric	Without Index	With Index
Initial search phase	18,859 nodes	1 node
Computational cost	661,062 DB Hits	623,252 DB Hits
Execution time	206 ms	191 ms

Table 9.9: Performance comparison of the Find Similar Chef query.

Consequently, despite the latency reduction achieved through this index, the intrinsic nature of the recommendation algorithm still requires an extensive traversal of the graph structure. To mitigate the impact of this computational cost on the application’s overall performance, we adopted an on-demand architectural pattern. Rather than computing and displaying this data by default upon loading a chef’s profile, the "Similar Chefs" feature is exclusively triggered by an explicit user request, specifically when a *Foodie* clicks the dedicated button.

9.3 Sharding

This section focuses on sharding. Although it was not ultimately implemented in the system, the following paragraphs outline the theoretical strategies and techniques that would have been employed had it been required.

9.3.1 MongoDB sharding

In MongoDB, we defined different sharding strategies based on how each collection is accessed by the application, specifically differentiating between user roles (Foodies and Chefs) and Recipes.

Foodies Collection

For the Foodies collection, an hashed sharding strategy was chosen based on the `username` field. Unlike the Chefs collection, Foodies are not subject to complex search operations, their interactions are primarily limited to registration, logging in, and managing personal lists. Consequently, this hashed approach is optimal: it ensures an even distribution of data and allows the system to perform highly efficient, targeted searches during authentication, rapidly locating a user without the need to broadcast queries across multiple shards.

Chefs Collection

For the Chefs collection, we have decided not to implement sharding at the moment, since the number of chefs in our application is currently limited. However, as the application grows, we plan to use a hashed sharding strategy on the `_id` field.

While this strategy may slightly increase the response time for a chef's login (since the database is forced to broadcast the authentication query across multiple shards), it heavily optimizes the system's most frequent action. When a *Foodie* clicks on a chef's profile from a recipe, fetching the details by `_id` remains extremely fast and direct. In conclusion, this design choice favors high-frequency read operations over occasional authentication tasks.

Recipes Collection

For the Recipes collection, we chose a hashed sharding strategy on the `_id` field. The vast majority of our database operations involve retrieving or displaying the details of a specific recipe using its `_id`.

We also considered sharding by `category` (e.g., "Desserts", "Main Dishes"), since users can filter recipes using this field. However, we discarded this approach for two main reasons. First, filtering by category is a less frequent operation than accessing a specific recipe's details, which relies on a direct search by `_id`. Second, sharding by category would create an uneven distribution of data, as some categories contain

significantly more recipes than others (for example, "Main Dishes" compared to "Drinks and Beverages"). Hashing the `_id` ensures that the workload is uniformly distributed across all shards, while guaranteeing that the system can immediately locate the target document during its most frequent operations.

9.3.2 Redis sharding

For Redis, we opted for a Sentinel Master-Slave architecture. Looking specifically at the query patterns for the Smart Fridge and Shopping List features, the workload is predominantly read-bound. Write operations, such as inserting or deleting items, are performed less frequently compared to routine read operations like checking the lists or loading inventory-based recipe recommendations. Since these functionalities are unlikely to generate massive write spikes, a Redis Cluster architecture is currently unnecessary, and no sharding has been implemented.

If these services grow in the future and start generating a lot of write operations, we could easily migrate to a Redis Cluster. In that case, we would use a sharding strategy based on key hashing. Specifically, we would use Redis "Hash Tags" to ensure that all data belonging to a single user is always routed to the same shard.

9.4 Virtual Machine Organization

This section describes how the database technologies are distributed across the available Virtual Machines. The deployment was specifically designed to balance the workload and ensure the system remains highly available.

9.4.1 Infrastructure Structure

The database infrastructure is distributed across the three virtual machines (VMs) provided, with the replicas strategically deployed as follows:

- **10.1.1.92:** Hosts the MongoDB Primary node and the Redis Master node.
- **10.1.1.91:** Hosts a MongoDB Secondary node and a Redis Replica (slave).
- **10.1.1.90:** Hosts a MongoDB Secondary node, a Redis Replica (Slave), and the standalone Neo4j instance.

This specific design was selected to ensure a robust load-balancing strategy, minimizing resource contention among concurrent database services.

9.4.2 Database VM Configurations

MongoDB

Replica Set Architecture For MongoDB document database, three replicas were implemented with the following configuration:

```
{ _id: 0, host: 'mongo_machine1:10.1.1.92', priority: 5 },  
{ _id: 1, host: 'mongo_machine2:10.1.1.91', priority: 2 },  
{ _id: 2, host: 'mongo_machine3:10.1.1.90', priority: 1 }
```

The assigned **priority** parameters are designed to handle the election process in case of a primary node failure. By allocating the maximum priority value (**priority: 5**) to machine **10.1.1.92** and the lowest (**priority: 1**) to machine **10.1.1.90**, the cluster explicitly favors the promotion of the secondary node to avoid overloading machine **10.1.1.90**, which manages the Neo4j engine and is consequently already subject to computationally intensive operations.

Write Concern and Read Preference To maximize throughput and align the database layer with the system’s performance requirements, the following policies have been implemented:

- **Write Concern: Write concern 1 (W1)** This configuration prioritizes fast write operations and low latency. It deliberately accepts a minimal risk of data loss in the rare event of a Primary node failure occurring before the asynchronous replication to the Secondaries is completed.
- **Read Preference: Nearest** This setting dynamically routes read operations to the cluster member with the lowest network latency. This effectively distributes the read load across the available infrastructure and minimizes query response times.

Redis

To ensure high availability and mitigate the risk of a Single Point of Failure within the Redis infrastructure, a Master-Replica architecture was implemented. The deployment consists of a single Master node hosted on machine **10.1.1.92**, alongside two Replica nodes on machines **10.1.1.91** and **10.1.1.90**. All Redis instances operate on port **6379**, while the Sentinel processes listen on port **7004**.

In fact, these Redis Sentinel processes are deployed across the infrastructure specifically to ensure fault tolerance. Sentinel constantly monitors the health of the nodes and, in the event of a Master failure, automatically triggers a failover process. This process elects one of the Replicas as the new Master, seamlessly maintaining service continuity.

Neo4j

Because the application’s data is highly connected and graph queries are computationally intensive, Neo4j is deployed as a single, standalone instance hosted on machine **10.1.1.90**. In fact, unlike document databases, graph databases rely on constantly navigating relationships between nodes. This makes them unsuitable for traditional sharding, because splitting relationships across different machines would cause severe performance drops.

9.5 CAP Theorem Considerations

According to the CAP theorem, the system architecture aligns with an AP (Availability and Partition Tolerance) model. As outlined in the non-functional requirements and reflected in our database design choices, the platform sustains a high volume of operations and is heavily read-oriented. Consequently, guaranteeing high availability is a fundamental requirement for the entire infrastructure.

Availability

In our architecture, high availability is systematically guaranteed through a combination of strategic data redundancy and asynchronous workload management, which together ensure high responsiveness.

Data Redundancy To prevent read bottlenecks, the system extensively relies on cross-database redundancy. At the graph level, the Smart Fridge feature caches recipe suggestions directly in Redis: when a user modifies their ingredients, the cached recipes are updated in-memory, bypassing Neo4j entirely and drastically reducing the query load on the graph database.

Internally, MongoDB leverages extensive data duplication to serve read operations as fast as possible, heavily denormalizing documents. For instance, a Chef document embeds previews of their 15 most recent recipes alongside aggregated metrics (e.g., total recipes, overall save count), while a Foodie profile inherently embeds previews of all their saved recipes. Furthermore, previews of pending recipes are safely duplicated within both the authoring chef's and the admin's documents.

Asynchronous Workload Management (LowLoadManager) The second key point of our availability strategy is the implementation of a custom *LowLoadManager* class (detailed in Section 7.7). This component has a straightforward goal: separating urgent write operations from secondary tasks. While critical updates are executed immediately, non-essential operations are postponed until the system load drops below 30%. We applied this strategy across three main patterns:

- **Counter Updates:** When a foodie saves or removes a recipe from their favorites, or permanently deletes their account, the system prioritizes the synchronous execution of the core operations (e.g., updating the embedded previews or securely removing the user profile). However, the subsequent recalculation of associated aggregated metrics (like a recipe's individual save count or a chef's overall popularity score) is delegated to the *LowLoadManager*. A temporarily delayed counter does not compromise the core user experience but saves critical computational resources during peak times.
- **Cross-Database Synchronization:** Synchronization between MongoDB and Neo4j is handled asynchronously. When a new recipe is approved or deleted, or when a chef (along with their recipes) is deleted, the changes are immediately

committed to MongoDB, which acts as the absolute source of truth. The *LowLoadManager* subsequently propagates these structural changes to the Neo4j graph. While this introduces a brief window of inconsistency where graph-based features might not reflect the absolute latest data, it drastically reduces the synchronous latency of write operations.

- **Lazy Resolution of Deleted Data:** To avoid heavy, cascading delete operations across the entire distributed system, we adopted a "lazy" clean-up strategy. When a recipe is deleted, it is proactively removed only from **Chefs** and **Recipes** collections. Stale references that might persist embedded in foodie profiles within their favorites, or cached in Redis within the Smart Fridge recipe suggestions, are not actively tracked down. Instead, if a user interacts with a stale preview, the application intercepts the resulting error and seamlessly removes the outdated reference on the fly.

Partition Tolerance

Partition tolerance ensures that the system continues to operate despite arbitrary network partitions or communication failures between its components. By design, our architecture ensures that the platform continues to serve requests even if specific nodes or databases become temporarily unreachable.

For example, if Neo4j experiences an outage or a network partition, the system remains operational by retrieving core recipe details from MongoDB and serving cached Smart Fridge recommendations directly from Redis. This strategy perfectly reflects the AP principles of the CAP theorem: rather than blocking operations or bringing the application offline, the architecture guarantees continuous availability and responsiveness to the user, consciously accepting temporary or partial data inconsistencies across the separated partitions.

9.6 Handling Inter-DB Consistency

Integrating three distinct database technologies within the same architecture inherently introduces the challenge of maintaining data consistency across multiple representations. Within our application, the storage roles are strictly partitioned based on the specific strengths of each database:

- **MongoDB** acts as the primary system of record, storing the comprehensive models for users (both Foodies and Chefs), recipes, and the administrative approval queues.
- **Neo4j** maintains a specialized, graph-oriented projection of this data, specifically isolating the interconnected network of chefs, recipes, and ingredients.
- **Redis** serves as the high-performance, in-memory layer, providing ultra-low latency access for dynamic, high-frequency features such as the Smart Fridge, the Shopping List, and cached recipe recommendations.

Neo4j

The main consistency challenge within the architecture involves MongoDB and Neo4j, since core entities like **Recipe** and **Chef** are distributed across both databases. MongoDB serves as the primary system of record containing the complete entity profiles. In contrast, Neo4j stores a lightweight projection of these entities (connected via the `mongo_id` field) that includes only the essential attributes needed to populate recipe previews, keeping the graph storage optimized and tailored to presentation needs.

To manage this duality without compromising performance, the system implements an *eventual consistency* approach driven by the **LowLoadManager**. Write operations that modify shared data are strictly executed synchronously on MongoDB. Upon success, a background task is delegated to the manager to propagate the structural changes to Neo4j asynchronously, during periods of low system load. In case of an error, such as Neo4j being temporarily unavailable, the execution fails and an error is logged. The task is then automatically re-added to the queue for a subsequent retry.

This design choice heavily prioritizes high availability and minimizes user-facing latency. By deferring the graph update, the system inevitably introduces a temporary window of inconsistency. For instance, when an administrator approves a new recipe, it becomes instantly accessible in MongoDB but might momentarily be missing from Neo4j-driven features, such as Smart Fridge recommendations. However, this perfectly aligns with the system's AP architecture, and absolute consistency is restored as soon as the pending synchronization task executes.

Redis

Because Redis stores data derived from Neo4j traversals and linked to MongoDB entities, the cache may occasionally reflect a stale application state. This occurs either because the cache was populated during a window of temporary inconsistency between Neo4j and MongoDB, or because the underlying entities were subsequently modified or deleted in the primary databases.

To mitigate these inter-database inconsistencies without introducing synchronous overhead, the system relies on three distinct recovery mechanisms:

- **State-Driven Cache Invalidation:** If the Smart Fridge cache is generated while Neo4j is temporarily out of sync with MongoDB, it might miss recently approved recipes. This inconsistency is naturally resolved whenever a user adds new ingredients and the existing cache is strictly invalidated. This forces a fresh traversal of the graph database, resynchronizing the user's recommendations with the latest data.
- **Lazy Eviction (Read-Repair):** If a recipe is deleted from MongoDB and Neo4j, its projection might temporarily persist as a stale reference within the Redis cache. To handle this, the system adopts a lazy, eventual consistency

approach. If a user attempts to view the details of an obsolete recipe, the application intercepts the resulting "not found" error from MongoDB and seamlessly removes the invalid entry from Redis on the fly.

- **TTL Expiration for Orphaned Data:** To maintain consistency when a Foodie permanently deletes their profile from MongoDB, the system avoids issuing synchronous delete commands to Redis. Instead, a Time-To-Live (TTL) mechanism is applied to all user-specific caching keys. Configured with a 15-day expiration period (refreshed upon interaction), this ensures that orphaned Smart Fridge and Shopping List data are automatically garbage-collected.

Chapter 10

Use of Artificial Intelligence in Development

It is important to highlight that, during the project's development, AI tools were used as a support mechanism rather than a replacement for human decision-making. All generated suggestions were carefully evaluated, adapted and integrated into the project according to the specific requirements. The use of AI contributed to improving development efficiency and code quality, while maintaining full control over the design and implementation choices.

10.1 Overview

10.1.1 Web Scraping

Given the structural complexity of the targeted websites (detailed in Chapter 5), AI support proved highly valuable during the data acquisition phase, primarily assisting in optimizing and debugging the Python scraping scripts to ensure a robust pipeline. This assistance seamlessly extended to the data cleaning phase (Section 5.1.2): due to the substantial volume of initially extracted data, comprehensive manual reviews were conducted and AI tools were employed to cross-validate the dataset, suggest normalization strategies, and detect subtle inconsistencies, facilitating the development of a clean and highly consistent dataset.

10.1.2 Support in Database Population

After the initial dataset was imported into MongoDB using MongoDB Compass, AI tools helped develop the Java scripts needed to populate and synchronize the rest of the system. First, some scripts updated the existing MongoDB collections by automatically calculating user favorites and chef statistics. Additionally, AI assistance made it easier to transfer data from MongoDB to Neo4j by helping to create the graph nodes, relationships, and performance indexes. Finally, these tools supported the writing of the code needed to load allowed ingredients and user lists into Redis from local files.

10.1.3 Support in Mockups Creation

Since the primary focus of this work is centered on backend architecture and database optimization, a physical user interface was not implemented. Instead, LLMs were used during the design phase to formalize conceptual mockups, providing a concrete

visualization of the system’s look and feel. This approach allowed us to map the abstract functional requirements directly onto tangible interface layouts. Finally, these mockups were carefully aligned with the implemented backend services and the underlying database design choices, ensuring perfect consistency between the conceptual user experience and the structured polyglot data models.

10.1.4 Java

Large Language Model (LLM) tools served as a valuable consultative resource during the architectural design of the background task management system. To handle asynchronous execution, we asked the LLM to identify the optimal Java constructs, leading to the implementation of a thread-safe `ConcurrentLinkedQueue` and the `@EnableScheduling` annotation. Furthermore, to enforce our custom low-load execution policy, we specifically queried the LLM for a native CPU monitoring built-in function. This highlighted the `OperatingSystemMXBean` interface, which we integrated into a custom handler to guarantee that task batches are processed exclusively when CPU utilization remains below the 30% threshold.

It also helped providing the correct regular expression to verify the preparation time fields within the recipe creation module:

```
1 {return this.prepTime.matches("^\\d+(\\.\\d+)?\\s+[a-zA-Z]+$");}
```

Finally, these tools significantly helped in the development workflow by assisting in the interpretation and resolution of complex debugging errors encountered within the IntelliJ IDE.

10.1.5 Support in Translation

To overcome linguistic barriers and ensure a high standard of professional communication, AI tools were utilized to support English translation and lexical refinement. This assistance facilitated the creation of more fluent and coherent documentation. Furthermore, this approach was extended to the source code documentation; LLM tools were employed to draft the Javadoc, helping to accurately translate complex technical concepts and correct grammatical inaccuracies, thereby enhancing the overall readability of the codebase.

10.1.6 Redis

LLM tools helped us with suggesting practical solutions such as using the `-raw` flag to properly render cached recipe suggestions in JSON format via the Redis CLI. Also its support was crucial during the infrastructure deployment phase: it played a useful role in diagnosing and resolving complex configuration errors while setting up the Master-Replica high-availability topology across the provided virtual machines.

10.1.7 MongoDB and Neo4j

Throughout the database optimization phase, LLM tools were utilized to refine, format, and troubleshoot complex queries across both MongoDB and Neo4j, strictly after they had been manually drafted. For MongoDB, this support was particularly valuable in correctly structuring the advanced mathematical operations required for the Chef Bayesian Ranking aggregation.

Furthermore, AI assistants aided our index performance analysis by suggesting appropriate evaluation utilities, such as the `PROFILE` keyword in Neo4j. Given the verbosity and complexity of these profiling outputs, the tools guided our interpretation of the execution plans, highlighting precisely which metrics to analyze in order to draw accurate conclusions and successfully boost query performance.